

Computer Science II

E2: approximate, multi-scale, and spatial stochastic simulation

Marco S. Nobile, Ph.D.

Bachelor's Degree in Engineering Physics

Ca' Foscari University of Venice

A.Y. 2025-2026



Università
Ca' Foscari
Venezia

Outline



- SSA (reprise)
- The Poisson distribution
- tau-leaping
- Choosing the tau value
- Common pitfalls and heuristics
- Towards SDEs
- Well-stirred systems?
- Reaction/Diffusion
- Next Subvolume Method
- Tackling different scales with hybrid simulation
- Harris' partitioned leaping algorithm and its variants



SSA (reprise)

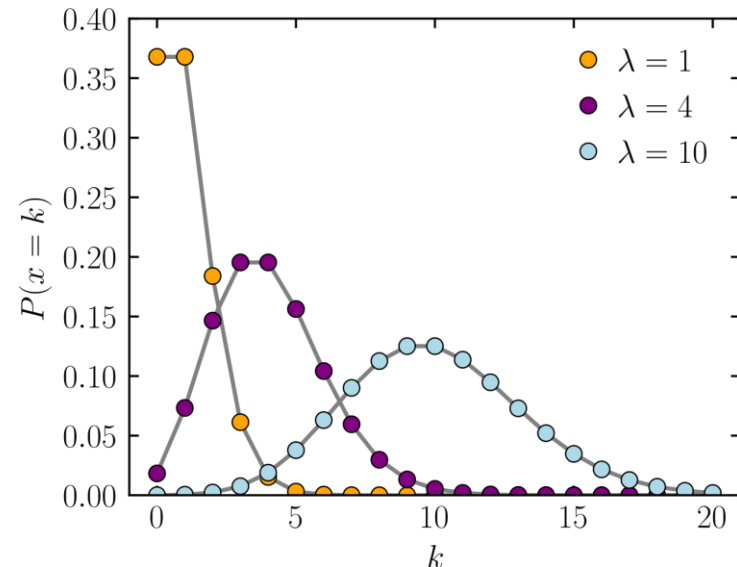
- Gillespie's SSA models the system as a Markov chain and performs the simulation by calculating **one reaction at a time**
- The time step of the simulation is calculated stochastically as $\tau = \frac{1}{a_0(\mathbf{X})} \ln \frac{1}{r_1}$ where a_0 is the sum of all propensities and r_1 is a random value from $\mathcal{U}[0,1)$
 - Remember that the **value of a propensity** function is **proportional to the amount of its reactants**
 - **The more the reactants, the smaller the time step, the slower the simulation**
- Is there a better (= faster) strategy to simulate a system of this kind?

The Poisson distribution

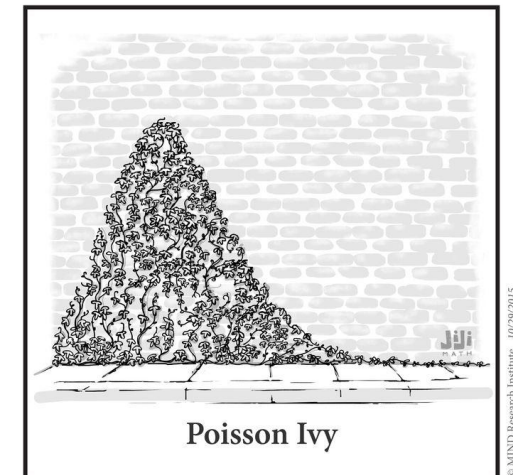
The Poisson Distribution (PD) is a **discrete probability distribution**

The PD expresses the probability of a **given number of independent events** occurring in a **fixed time interval** with a constant rate

- e.g., the number of decay events from a radioactive source in a given time interval, the number of text messages per hour or... the number of chemical reactions happening in specific time unit



k is the number of occurrences
 λ is the rate of the occurrences

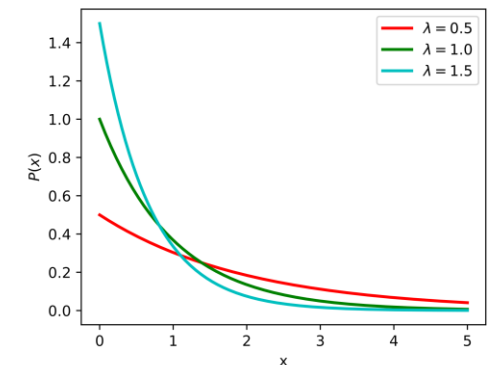


Poisson processes



- Model sequence of **independent events** happening randomly over time
- The average **rate** λ of events per unit time is **constant**
- **The number of events** in the time interval of length t follows a Poisson distribution:

$$P(N(t) = k) = \frac{(\lambda t)^k e^{-\lambda t}}{k!}$$



Generating Poisson random numbers



- Generate exponential interarrival times
- Count how many events occur in a fixed time interval
- The number of events is Poisson-distributed

Generating Poisson numbers



- The simplest algorithm to generate random numbers from a PD is due to Knuth (see [\[Knuth, *The Art of Computer Programming*, 1997\]](#)):

```
init:
    Let  $L \leftarrow \exp(-\lambda)$ ,  $k \leftarrow 0$  and  $p \leftarrow 1$ .
do:
     $k \leftarrow k + 1$ .
    Generate uniform random number  $u$  in  $[0,1]$  and let  $p \leftarrow p \times u$ .
while  $p > L$ .
return  $k - 1$ .
```

- Problem 1: complexity is proportional to λ (does not scale well for large values)
- Problem 2: $e^{-\lambda}$ can become very small extremely fast!
- In general, Knuth's algorithm should never be used for $\lambda > 30$

Poisson random numbers by rejection sampling

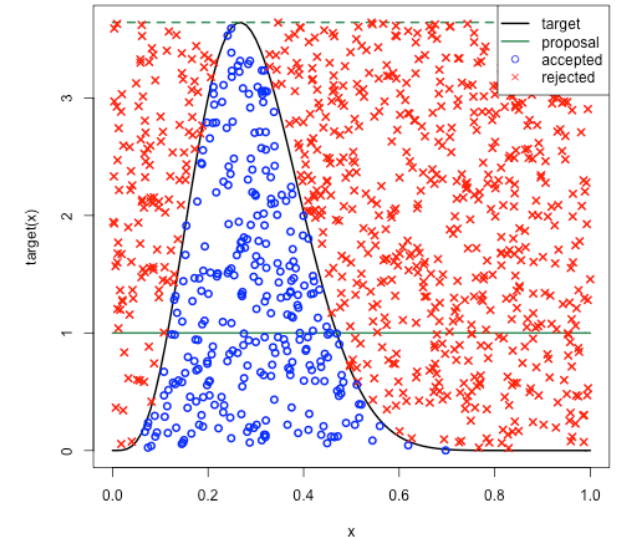


- There are several improvements, but all lead to some sort of issues ([John D. Cook's analysis](#))
- A good approach is the so-called **Rejection Sampling (RS)**
- Rejection sampling – aka acceptance-rejection – is a technique to **generate samples from any "difficult" distribution** with analytical density $f(x)$

How does RS work?

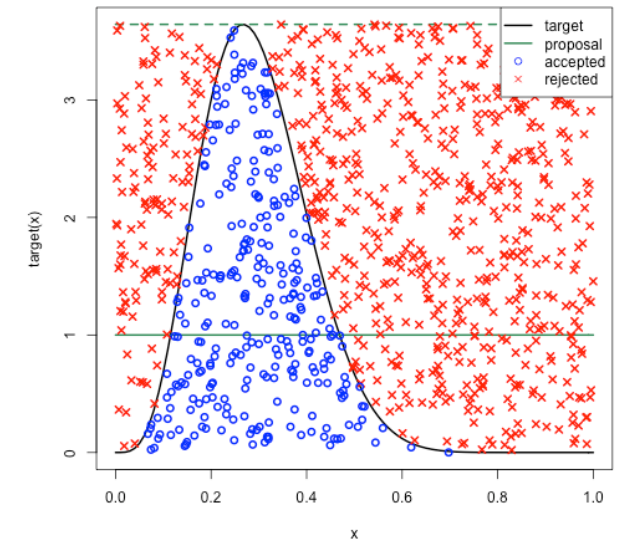
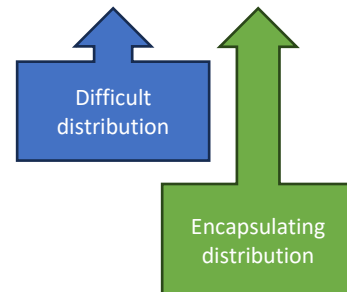


- The idea of RS is to exploit random values from some "**proxy**" distribution $g(x)$ that we can **easily** take samples from
 - E.g., the **uniform** distribution $\mathcal{U}[a, b]$, that is, $g(x) = \frac{1}{b-a}$ (solid green line in the figure)
 - The proxy distribution must "**encapsulate**" $f(x)$, that is, $f(x) < Cg(x), \forall x$
 - We can determine C by calculating the maximum of $f(x)$
- RS algorithm:
 - Randomly choose $u \in \mathcal{U}[a, b]$
 - Accept u with probability $f(u)/Cg(u)$
 - Cleanse and repeat



How does RS work?

- The idea of RS is to exploit random values from some "**proxy**" distribution $g(x)$ that we can **easily** take samples from
 - E.g., the **uniform** distribution $\mathcal{U}[a, b]$, that is, $g(x) = \frac{1}{b-a}$ (solid green line in the figure)
 - The proxy distribution must "**encapsulate**" $f(x)$, that is, $f(x) < Cg(x), \forall x$
 - We can determine C by calculating the maximum of $f(x)$
- RS algorithm:
 - Randomly choose $u \in \mathcal{U}[a, b]$
 - Accept u with probability $f(u)/Cg(u)$
 - Cleanse and repeat



Subtleties of RS



- Remember that a PD is **discrete** and only generates natural numbers (→ round down to the next lower integer)
- A poor choice of $g(x)$ can lead to **infinite loops** or **discarding many samples**
- Using a **Gaussian** for $g(x)$ as in the code on the left mitigates the waste of samples, but requires to calculate $n!$ which can lead to **overflow errors**
- Take home message: there's no free lunch, guys

```
c = 0.767 - 3.36/lambda
beta = PI/sqrt(3.0*lambda)
alpha = beta*lambda
k = log(c) - lambda - log(beta)

forever
{
    u = random()
    x = (alpha - log((1.0 - u)/u))/beta
    n = floor(x + 0.5)
    if (n < 0)
        continue
    v = random()
    y = alpha - beta*x
    lhs = y + log(v/(1.0 + exp(y))^2)
    rhs = k + n*log(lambda) - log(n!)
    if (lhs <= rhs)
        return n
}
```



τ -leaping [1/3]

- Given the state $\mathbf{x}$ of the system, we can denote by $K_j(\tau, \mathbf{x}, t)$ the number of times the reaction R_j would be fired in the time interval $[t, t + \tau)$
- For arbitrary values of τ , calculating the vector $\mathbf{K} = (K_1, \dots, K_M)$ would be **as difficult as solving the CME** $\rightarrow$ not feasible
- However, if the τ is **reasonably small** so that propensity functions "**will not suffer an appreciable change**" then it is possible to **approximate** $K_j(\tau, \mathbf{x}, t)$ with a **Poisson** random variable with mean and variance $a_j(\mathbf{x})\tau$



τ -leaping [2/3]

- Let us assume that the, in a given interval τ , **propensities functions change less than a given amount ϵ**
- Then, instead of simulating all chemical reactions (= SSA), we calculate **how many events happen** in $[t, t + \tau)$ using a **Poisson distribution**
- This way, we take **longer steps** and "aggregate" many reactions with short waiting time: this is the **τ -leaping algorithm**
- See [[Gillespie, J Chem Phys 2001](#)]



τ -leaping [3/3]

- The so-called "leap condition" must be satisfied to use τ -leaping:

Basic τ -Leap Method: Choose a value for τ that satisfies the Leap Condition; i.e., a temporal leap by τ will result in a state change $\boldsymbol{\lambda}$ which is such that, for every reaction channel R_j , $|a_j(\mathbf{x} + \boldsymbol{\lambda}) - a_j(\mathbf{x})|$ is “effectively infinitesimal.” Generate for each $j = 1, \dots, M$ a sample value k_j of the Poisson random variable $\mathcal{P}(a_j(\mathbf{x}), \tau)$, and compute $\boldsymbol{\lambda} = \sum_j k_j \mathbf{v}_j$. Finally, effect the leap by replacing t by $t + \tau$ and $\mathbf{x}$ by $\mathbf{x} + \boldsymbol{\lambda}$.

- For each possible reaction R_m , we calculate the **number of events** (=reactions) happening by generating $K_j \sim \text{Poisson}(a_j(\mathbf{x})\tau)$
- When we have calculated the number of events **for every reaction** R_j in the system, we can use the state change vectors $\mathbf{v}_j$ to **update the state** and iterate

A simple example

Simple enzyme kinetics: $S + E \xrightarrow{c_1=0.1} ES, ES \xrightarrow{c_2=1} P$

Initial state $\mathbf{X}(0)$: $S = 100, E = 10, ES = P = 0$

Simulation with $\tau = 0.1$

- Step 0
 - $a_1 = c_1 \cdot S \cdot E = 100, a_2 = c_2 \cdot ES = 0$
 - $K_1 = \text{Poisson}(a_1(\mathbf{x})\tau) = \text{Poisson}(\lambda = 10) = 5, K_2 = \text{Poisson}(a_2(\mathbf{x})\tau) = 0$
 - State $\mathbf{X}(\tau)$: $S = 95, E = 5, ES = 5, P = 0$
- Step 1
 - $a_1 = c_1 \cdot S \cdot E = 47.5, a_2 = c_2 \cdot ES = 5$
 - $K_1 = \text{Poisson}(a_1(\mathbf{x})\tau) = \text{Poisson}(\lambda = 4.75) = 2, K_2 = \text{Poisson}(a_2(\mathbf{x})\tau) = \text{Poisson}(\lambda = 0.5) = 1$
 - State $\mathbf{X}(2\tau)$: $S = 93, E = 3, ES = 6, P = 1$
- Ecc...

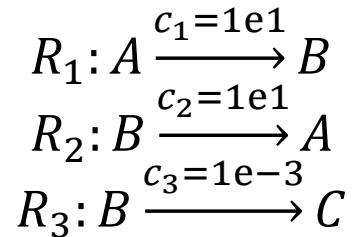
Random value!



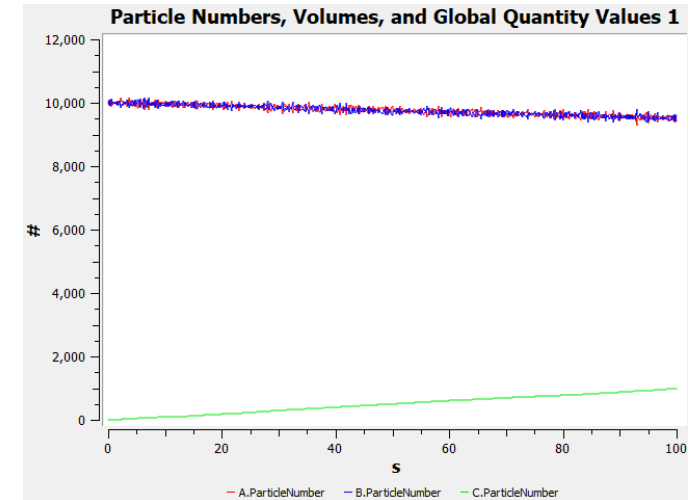
Performance



- Consider the following system having three chemical species involved in three reactions:



- The initial amounts are $A = 1e4, B = 1e4, C = 0$
- The computational effort is mainly spent to simulate reactions R_1 and R_2
 - Simulation time SSA: 2.93 s $\leftarrow$ SSA calculates *every* tiny simulation step
 - Simulation time τ -leaping: 0.28 s $\leftarrow$ takes longer jumps!





τ -leaping issues

- τ -leaping introduces several complications:
 1. Choosing τ
 2. Fast reactions can easily invalidate the leap condition (related to #1)
 3. Multiple reactions consuming the same reactants can lead to non valid states
 4. Poisson numbers generation is computationally intensive



τ -leaping issues

- τ -leaping introduces several complications:
 1. **Choosing τ**
 2. **Fast reactions can easily invalidate the leap condition** (related to #1)
 3. Multiple reactions consuming the same reactants can lead to non valid states
 4. Poisson numbers generation is computationally intensive



Efficient step selection

- One option for τ selection was proposed by [\[Cao et al., J Chem Phys 2006\]](#)
- Similarly to adaptive step size in ODE solution (you will learn about that soon)
Cao proposes to use a **tolerance** ϵ
 - The idea is to **bound the relative change** of each species X_i by ϵ/g_i
 - g_i is a quantity that depends on the **rate of change** of species S_i and is generally proportional to the highest order event rate
 - The paper suggests a tolerance $\epsilon = 0.03$

Calculating τ with Cao's algorithm

- Calculate the auxiliary values:

$$\mu_i(\mathbf{x}) = \sum_j v_{ij} a_j(\mathbf{x})$$
$$\sigma_i^2(\mathbf{x}) = \sum_j v_{ij}^2 a_j(\mathbf{x})$$

- For each variable X_i calculate the highest order event and obtain g_i
- Finally:

$$\tau = \min_i \left\{ \frac{\max\left\{\frac{\epsilon X_i}{g_i}, 1\right\}}{|\mu_i(\mathbf{x})|}, \frac{\max\left\{\frac{\epsilon X_i}{g_i}, 1\right\}^2}{\sigma_i^2(\mathbf{x})} \right\}$$



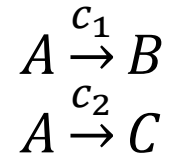
τ -leaping issues

- τ -leaping introduces several complications:
 1. Choosing τ
 2. Fast reactions can easily invalidate the leap condition (related to #1)
 3. **Multiple reactions consuming the same reactants can lead to non valid states**
 4. Poisson numbers generation is computationally intensive



An apparently harmless scenario

- Consider the following reactions:



with initial conditions: $A = 1, B = C = 0$

- The propensities are:

$$\begin{array}{l} a_1 = c_1 \cdot A \\ a_2 = c_2 \cdot A \end{array}$$

- Both reactions can, in principle, lead to a **non zero number of reactions firing**
 - That implies that at least 2 molecules of A would be removed!
 - In the next iteration, we would have -1 molecules of A, which is **impossible**



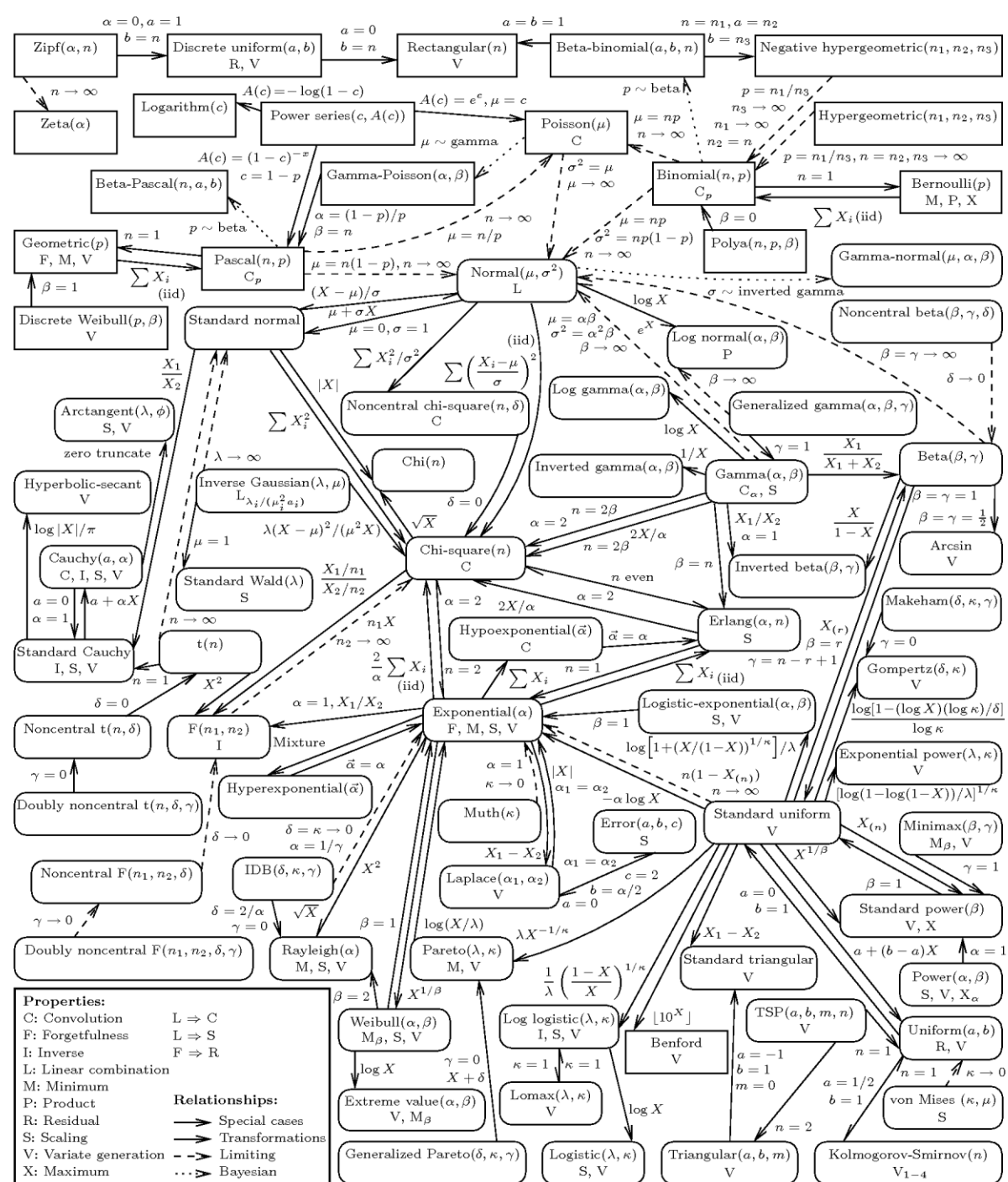
Critical reactions approach

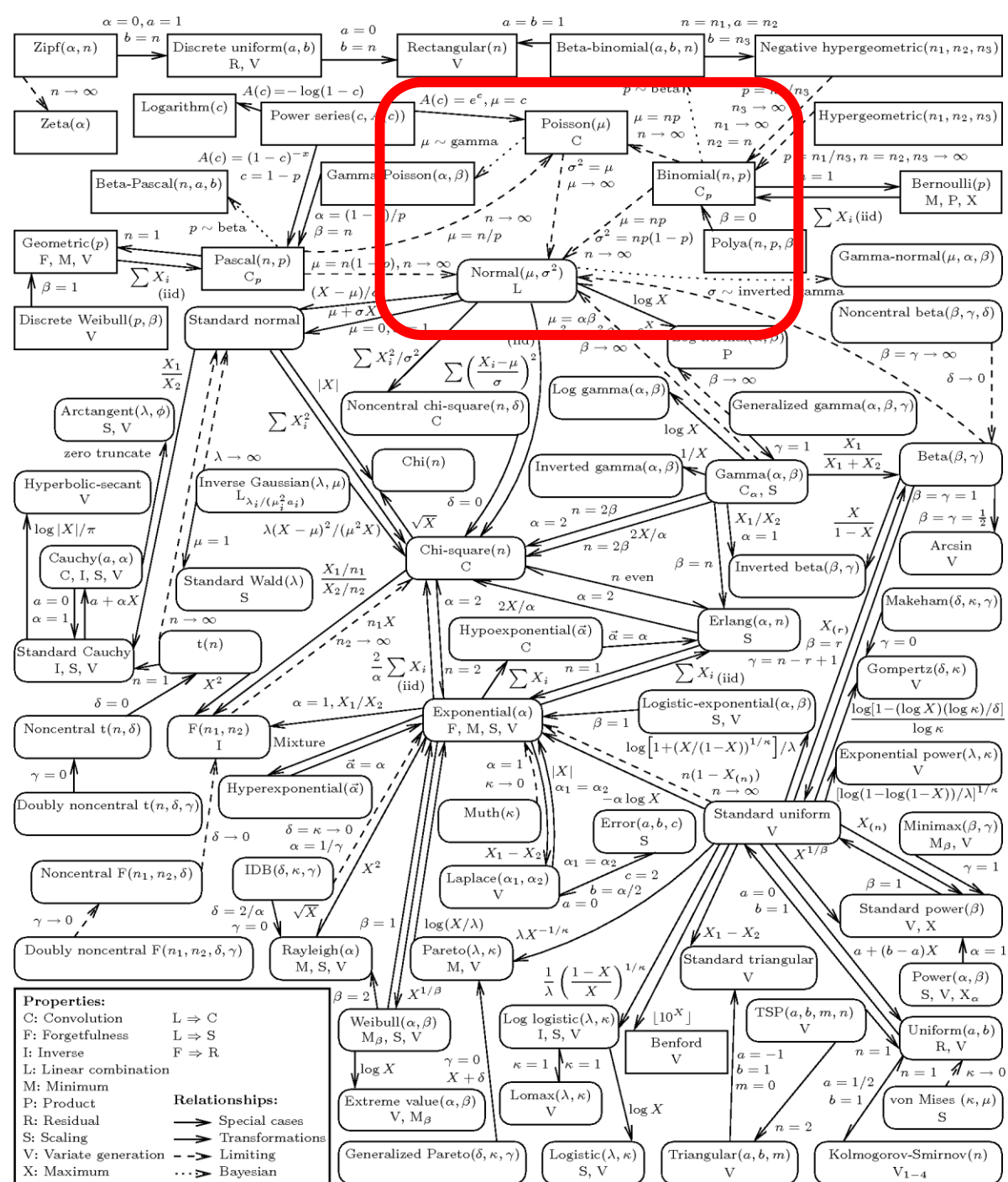
- One possible trick to tackle the problem of negative values is to **flag reactions as *critical*** if the number of reactants is not sufficient to **fire more than θ_c times** in the time interval
 - In each iteration, **only one critical reaction** is allowed to fire
 - Proposed by [\[Cao et al., J Chem Phys 2005\]](#), who suggested $\theta_c = 10$
- Of course, θ_c is a heuristic, it can happen that **multiple non-critical reactions fire simultaneously** and consume more reactants than those available
 - When an invalid state is reached, everything breaks down, so **we must fix that**
 - One possibility is to **rollback** to the last valid state, **reduce the length of τ** (e.g., taking $\tau = \tau/2$), and try again
 - Of course, this might induce a loop and the **performance would be affected**



τ -leaping issues

- τ -leaping introduces several complications:
 1. Choosing τ
 2. Fast reactions can easily invalidate the leap condition (related to #1)
 3. Multiple reactions consuming the same reactants can lead to non valid states
 4. **Poisson numbers generation is computationally intensive**







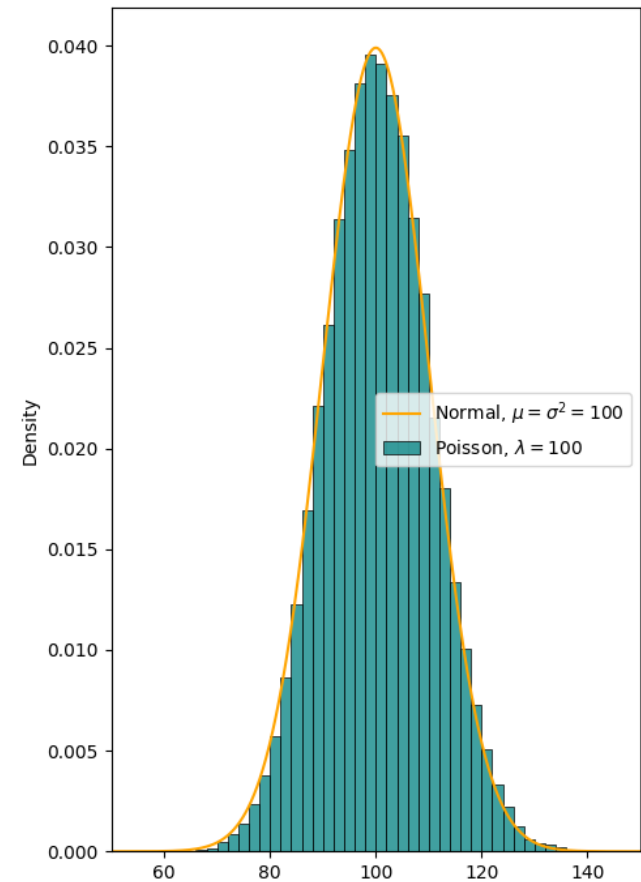
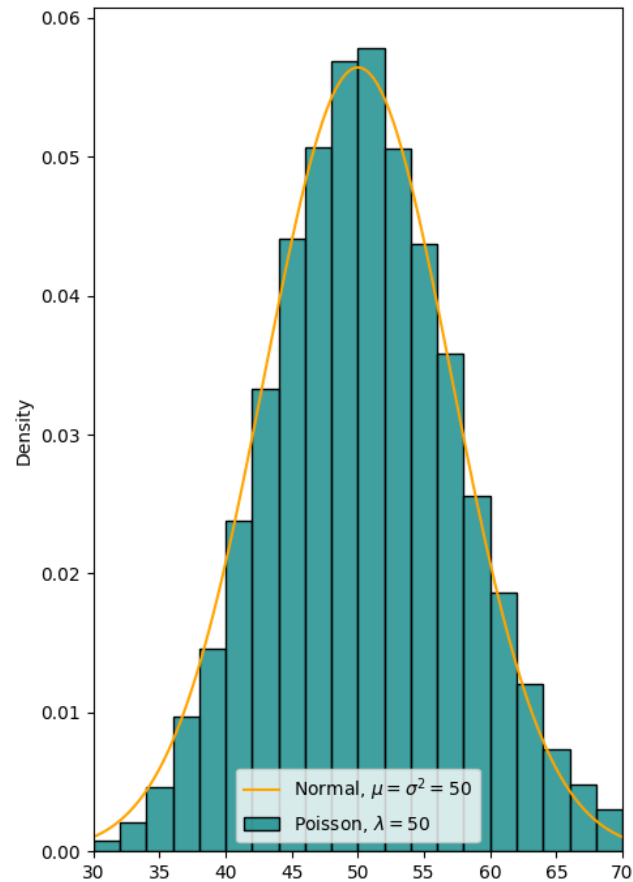
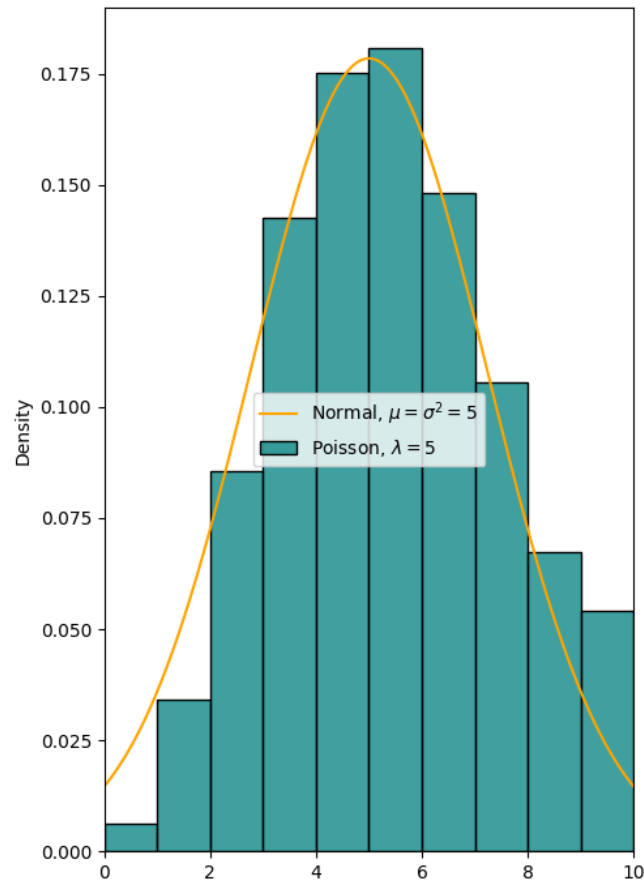
Approximating Poisson with normal

- If the value for the mean $\lambda > 10$ and $\sigma^2 = \lambda$, then the random variable is **normally distributed** and can be approximated by a Gaussian
- We need to apply a **continuity correction**, because the normal distribution is real valued and Poisson is discrete:

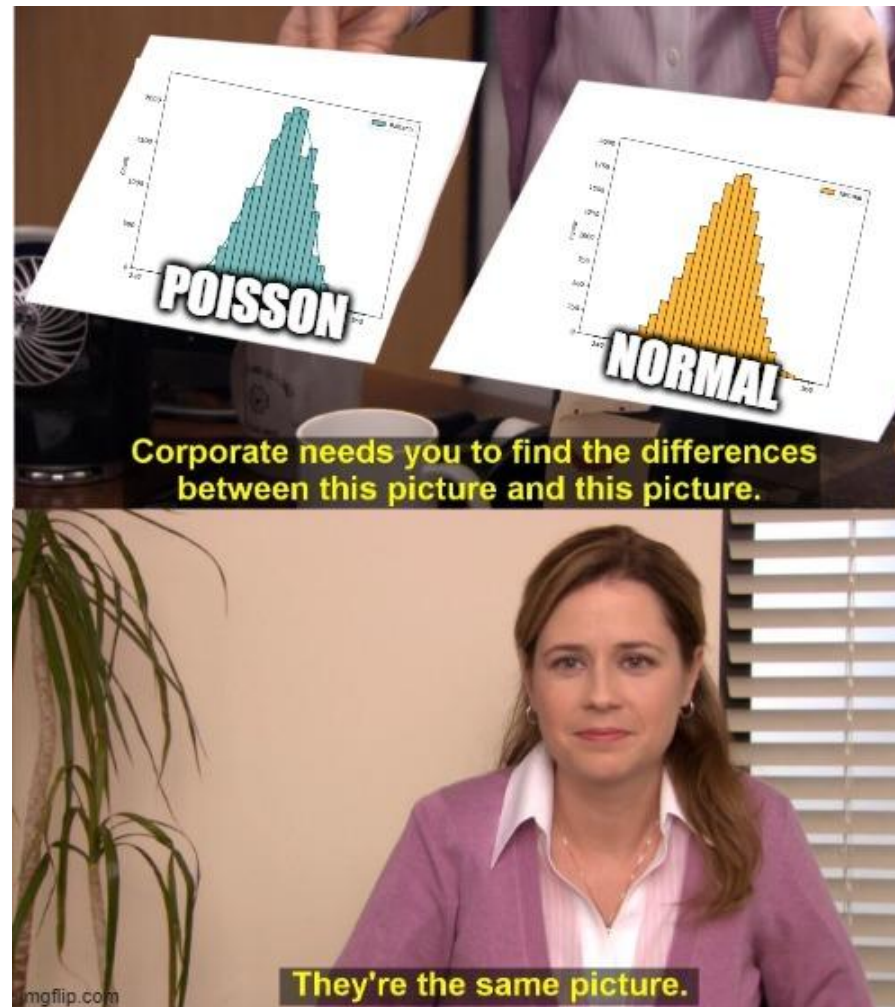
$$P(X \leq x) = P(X < x + 1) \approx P\left(Y \leq x + \frac{1}{2}\right)$$

where Y is normally distributed with $\mu = \sigma^2 = \lambda$

Approximating Poisson with Gaussian



In short





One final word about τ -leaping

- For systems characterized by **many fast reactions**, calculating the **Poisson numbers for all reactions** can be **computationally expensive**
 - So expensive that aiming for "long" steps would be a huge waste of time!
 - In these cases, it could be much more efficient to **temporarily switch to SSA instead**
 - It is like "inverse stiffness"
- Cao *et al.* suggested the following heuristic to switch to SSA:

$$\frac{\gamma}{a_0(\mathbf{x})} < \tau$$

with $\gamma = 10$

Relationship with SDEs



The chemical Langevin equation

J. Chem. Phys. 113, 297 (2000); <https://doi.org/10.1063/1.481811>

Daniel T. Gillespie

IV. AN APPROXIMATING CHEMICAL LANGEVIN EQUATION

Suppose the system's state $\mathbf{X}(t)$ at the current time t is known to be $\mathbf{x}_t$. Let $K_j(\mathbf{x}_t, \tau)$, for any $\tau > 0$, be the number of R_j reactions that occur in the subsequent time interval $[t, t + \tau]$. Since each of those reactions will increase the S_i population by ν_{ji} , the number of S_i molecules in the system at time $t + \tau$ will be

$$X_i(t + \tau) = x_{ti} + \sum_{j=1}^M K_j(\mathbf{x}_t, \tau) \nu_{ji} \quad (i = 1, \dots, N). \quad (16)$$

$K_j(\mathbf{x}_t, \tau)$ is of course a *random variable*. To compute it for arbitrary $\tau > 0$ would be quite as difficult as solving the master equation (6). But we can obtain an excellent *approximation* to $K_j(\mathbf{x}_t, \tau)$ rather easily if we impose the following conditions.

Relationship with SDEs

Condition (i): Require τ to be *small* enough that the change in the state during $[t, t + \tau]$ will be so slight that none of the propensity functions changes its value “appreciably;” i.e., the propensity functions then satisfy

$$a_j(\mathbf{X}(t')) \equiv a_j(\mathbf{x}_t), \quad \forall t' \in [t, t + \tau], \quad \forall j \in [1, M]. \quad (17)$$

Of course, we could easily satisfy Eq. (17) if we simply took τ to be so small that *zero* reactions would be likely to occur in $[t, t + \tau]$. But usually we need not be that extreme. Since the propensity functions generally depend on non-negative integer powers of the molecular species populations, and the latter practically never change by more than two molecules in any one reaction event, then we need only require that the likely number of reactions occurring in $[t, t + \tau]$ be much smaller than the population of the least populous reactant species. So condition (i) can *always* be satisfied if all the reactant molecule populations are sufficiently large compared to 1.

Since the reactions that occur in the time interval $[t, t + \tau]$ do not appreciably change the values of any of the propensity functions, then all reaction events occurring in the time interval $[t, t + \tau]$ will be essentially *independent* of each other. So $K_j(\mathbf{x}_t, \tau)$ will simply be the number of times reaction channel R_j would occur in a duration τ if its propensity function remained constant at the value $a_j(\mathbf{x}_t)$. In light of our fundamental premise (2), this means, as is shown in Appendix A, that each $K_j(\mathbf{x}_t, \tau)$ will be a *statistically independent Poisson* random variable, $\mathcal{P}_j(a_j(\mathbf{x}_t), \tau)$. So the net effect of condition (i) is to allow Eq. (16) to be approximated by

$$X_i(t + \tau) = x_{ti} + \sum_{j=1}^M \nu_{ji} \mathcal{P}_j(a_j(\mathbf{x}_t), \tau) \quad (i = 1, \dots, N). \quad (18)$$

We know that: the
leap condition!

We know that too:
tau-leaping!

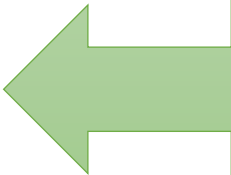
Relationship with SDEs

Next we impose a second condition on τ .

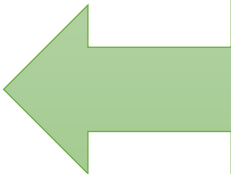
Condition (ii): Require τ to be *large* enough that the expected number of occurrences of each reaction channel R_j in $[t, t + \tau]$ be much larger than 1, i.e., (see Appendix A)

$$\langle \mathcal{P}_j(a_j(\mathbf{x}_t)), \tau \rangle = a_j(\mathbf{x}_t) \tau \gg 1, \quad \forall j \in [1, M]. \quad (19)$$

This condition obviously runs counter to condition (i), and it may very well happen that both conditions cannot be satisfied simultaneously; in that case, the scheme being devised here will fail. But there will be many practical circumstances in which conditions (i) and (ii) can be simultaneously satisfied. As with condition (i), satisfaction of condition (ii) is greatly facilitated by having large molecular population numbers: Since $a_j(\mathbf{x}_t)$, through the combinatorial function $h_j(\mathbf{x}_t)$, will typically be proportional to one or more components of $\mathbf{x}_t$, sufficiently large molecular populations can make it is possible for the inequalities (19) to hold even when τ is quite small.



Mh, that's a new one:
consider a τ such that a **large number of reactions** will occur



That would clearly conflict with the leap condition **unless we have a very large molecular population**

Relationship with SDEs

As explained in Appendix A, condition (ii), or more precisely the inequality (19), allows us to *approximate* each *Poisson* random variable $\mathcal{P}_j(a_j(\mathbf{x}_t), \tau)$ by a *normal* random variable with the same mean and variance. That brings Eq. (18) into the form

$$X_i(t + \tau) = x_{ti} + \sum_{j=1}^M \nu_{ji} \mathcal{N}_j(a_j(\mathbf{x}_t) \tau, a_j(\mathbf{x}_t) \tau) \\ (i = 1, \dots, N), \quad (20)$$

where $\mathcal{N}(m, \sigma^2)$ denotes the normal random variable with mean m and variance σ^2 . Notice that in replacing the *integer Poisson* random variables in Eq. (18) by the *real normal* random variables in Eq. (20), we in effect convert the molecular populations X_i from discretely changing integer variables to continuously changing real variables. Notice also that the M normals $\mathcal{N}_j$ in Eq. (20) will be *statistically independent*, this in consequence of the statistical independence of the M Poisson's $\mathcal{P}_j$ in Eq. (18).



**Normal approximation
of Poisson distribution**

Relationship with SDEs

The linear combination theorem for normal random variables,

$$\mathcal{N}(m, \sigma^2) = m + \sigma \mathcal{N}(0, 1),$$

can now be invoked to bring Eq. (20) into the form

$$\begin{aligned} X_i(t + \tau) = & x_{ti} + \sum_{j=1}^M v_{ji} a_j(\mathbf{x}_t) \tau \\ & + \sum_{j=1}^M v_{ji} [a_j(\mathbf{x}_t) \tau]^{1/2} \mathcal{N}_j(0, 1) \quad (i = 1, \dots, N), \end{aligned} \quad (21)$$

where again the M normals $\mathcal{N}_j(0, 1)$ are all statistically independent. We conclude by making some purely notational changes in this result: First, let us regard any time interval τ that satisfies *both* conditions (i) and (ii) as a *macroscopic infinitesimal*, and denote it simply by dt . Second, let us write the “unit normal” random variable $\mathcal{N}_j(0, 1)$ as $\mathcal{N}_j(t)$, with the understanding that $\mathcal{N}_j(t)$ and $\mathcal{N}_{j'}(t')$ will be statistically independent if either $j \neq j'$ or $t \neq t'$. And finally, let us recall that $\mathbf{x}_t$ stands for $\mathbf{X}(t)$. Equation (21) becomes

$$\begin{aligned} X_i(t + dt) = & X_i(t) + \sum_{j=1}^M v_{ji} a_j(\mathbf{X}(t)) dt \\ & + \sum_{j=1}^M v_{ji} a_j^{1/2}(\mathbf{X}(t)) \mathcal{N}_j(t) (dt)^{1/2} \\ & (i = 1, \dots, N). \end{aligned} \quad (22)$$

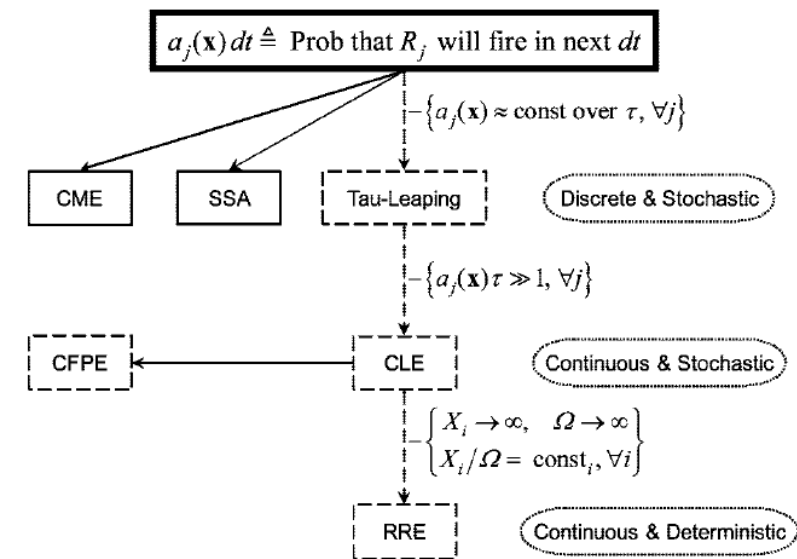
As explained in Appendix B, Eq. (22) has the canonical form of a “standard-form Langevin equation” for a multivariate continuous Markov process. Equation (22) thus implies the equivalent “white-noise form” Langevin equation

$$\begin{aligned} \frac{dX_i(t)}{dt} = & \sum_{j=1}^M v_{ji} a_j(\mathbf{X}(t)) + \sum_{j=1}^M v_{ji} a_j^{1/2}(\mathbf{X}(t)) \Gamma_j(t) \\ & (i = 1, \dots, N), \end{aligned} \quad (23)$$

where the $\Gamma_j(t)$ are temporally uncorrelated, statistically independent Gaussian white noises. Equation (22) also implies

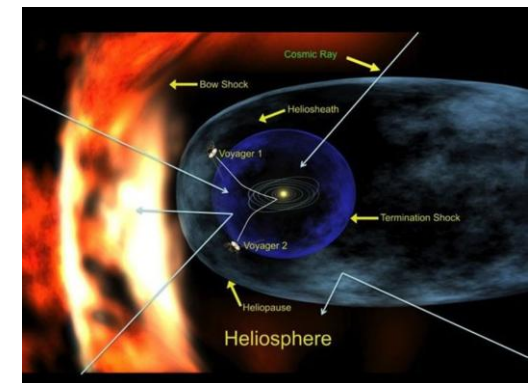
(see Appendix B) that the singly conditioned density function for $\mathbf{X}(t)$ obeys the (forward) Fokker–Planck equation

$$\begin{aligned} \frac{\partial}{\partial t} P(\mathbf{x}, t | \mathbf{x}_0, t_0) = & - \sum_{i=1}^N \frac{\partial}{\partial x_i} \left[\left(\sum_{j=1}^M v_{ji} a_j(\mathbf{x}) \right) P(\mathbf{x}, t | \mathbf{x}_0, t_0) \right] \\ & + \frac{1}{2} \sum_{i=1}^N \frac{\partial^2}{\partial x_i^2} \left[\left(\sum_{j=1}^M v_{ji}^2 a_j(\mathbf{x}) \right) P(\mathbf{x}, t | \mathbf{x}_0, t_0) \right] \\ & + \sum_{\substack{i, i'=1 \\ i < i'}}^N \frac{\partial^2}{\partial x_i \partial x_{i'}} \left[\left(\sum_{j=1}^M v_{ji} v_{ji'} a_j(\mathbf{x}) \right) P(\mathbf{x}, t | \mathbf{x}_0, t_0) \right]. \end{aligned} \quad (24)$$



More about SDEs at the end

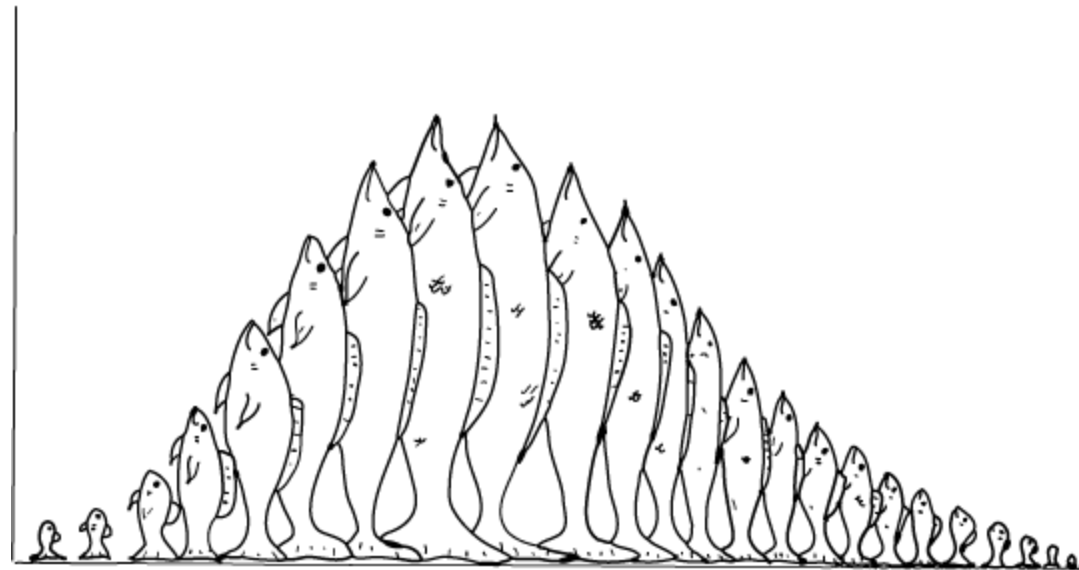
- There will be a seminar in December from Leone Bacciu, «Borsista di ricerca» within the PNRR project «SDEGnO» (SDEs and Global Optimization)
- SDEGnO is joint project with Milano-Bicocca and the Milano section of INFN (Istituto Nazionale di Fisica Nucleare) about the **prediction of cosmic rays propagation** within the heliosphere (useful to plan future expeditions)
- Extremely **complex physical model based on SDEs**, simulated on GPUs, and calibrated with computational intelligence methods



The Poisson distribution



Poisson Distribution



$$P\{x=i\} = e^{-\lambda} \cdot \frac{\lambda^i}{i!}$$

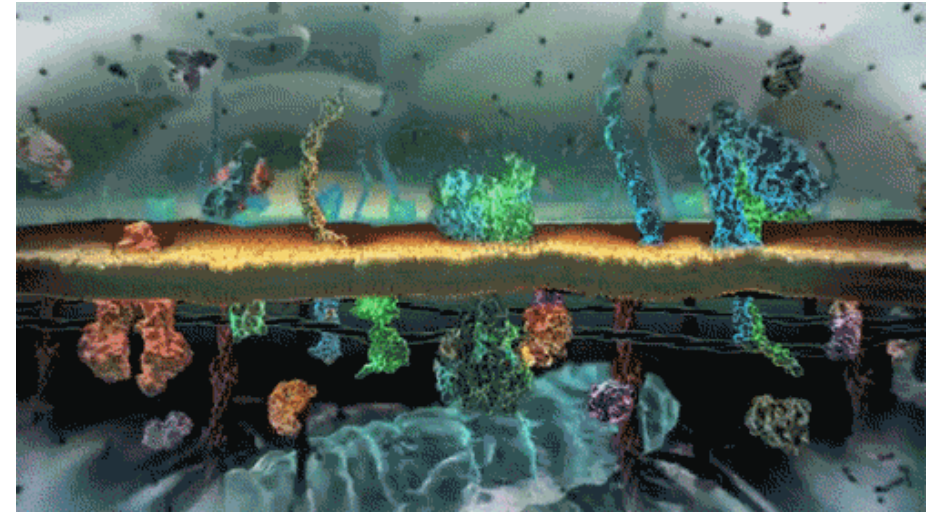
NICO

© 2006, Nico Nell

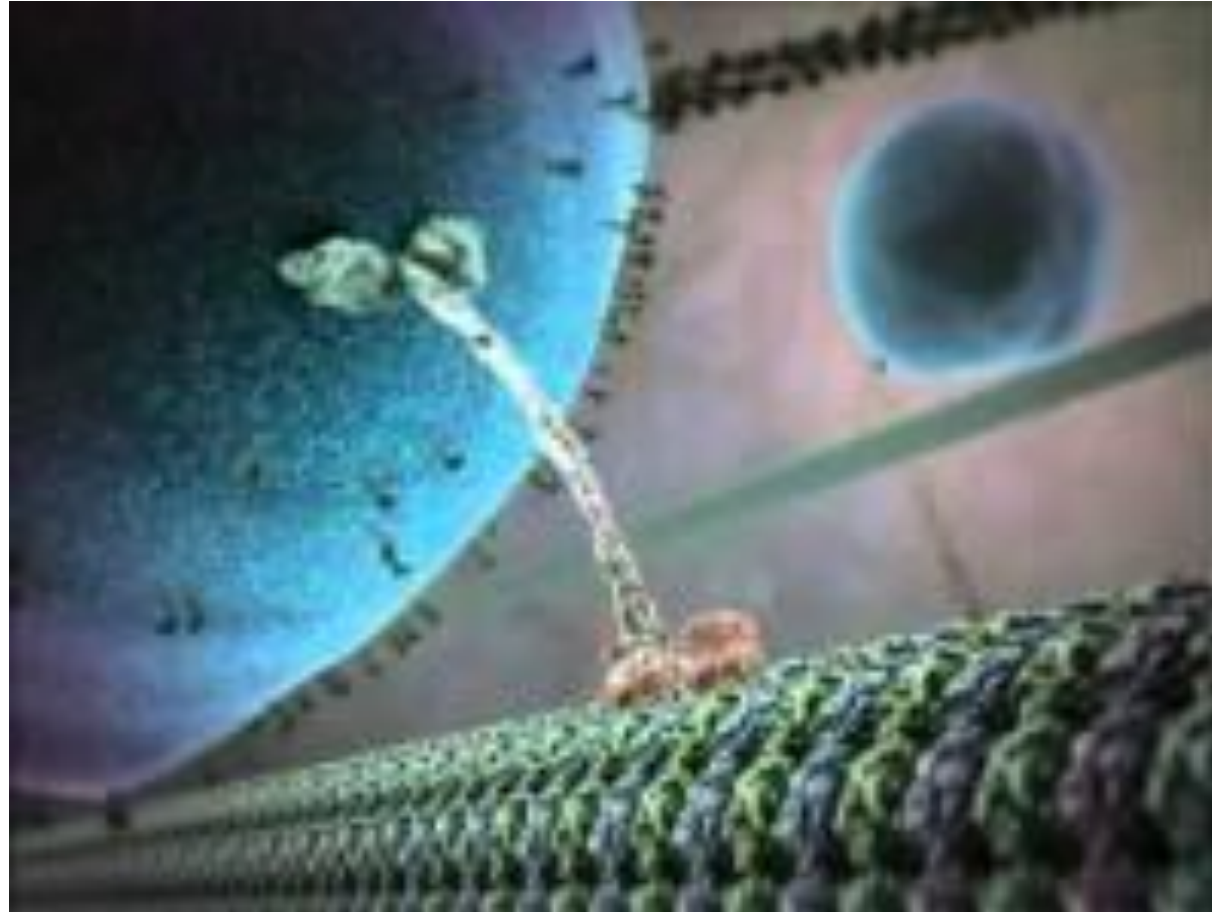
If you aren't laughing now then you passed the "I'm not a nerd test"

SSA (Gillespie's algorithm)

- We can use SSA to stochastically simulate the dynamic behavior of a chemical system whose reactants are **well-stirred**
 - That is, the molecules are **homogeneously distributed** in the space
- However, often (bio)molecules have **inhomogeneous spatial distribution**
 - Think of **membrane proteins**: they are anchored and not going anywhere!
 - Or consider **signal molecules**, traveling from the membrane to the nucleus
 - In this case, **SSA should not be applied** for a **key assumption would be violated**

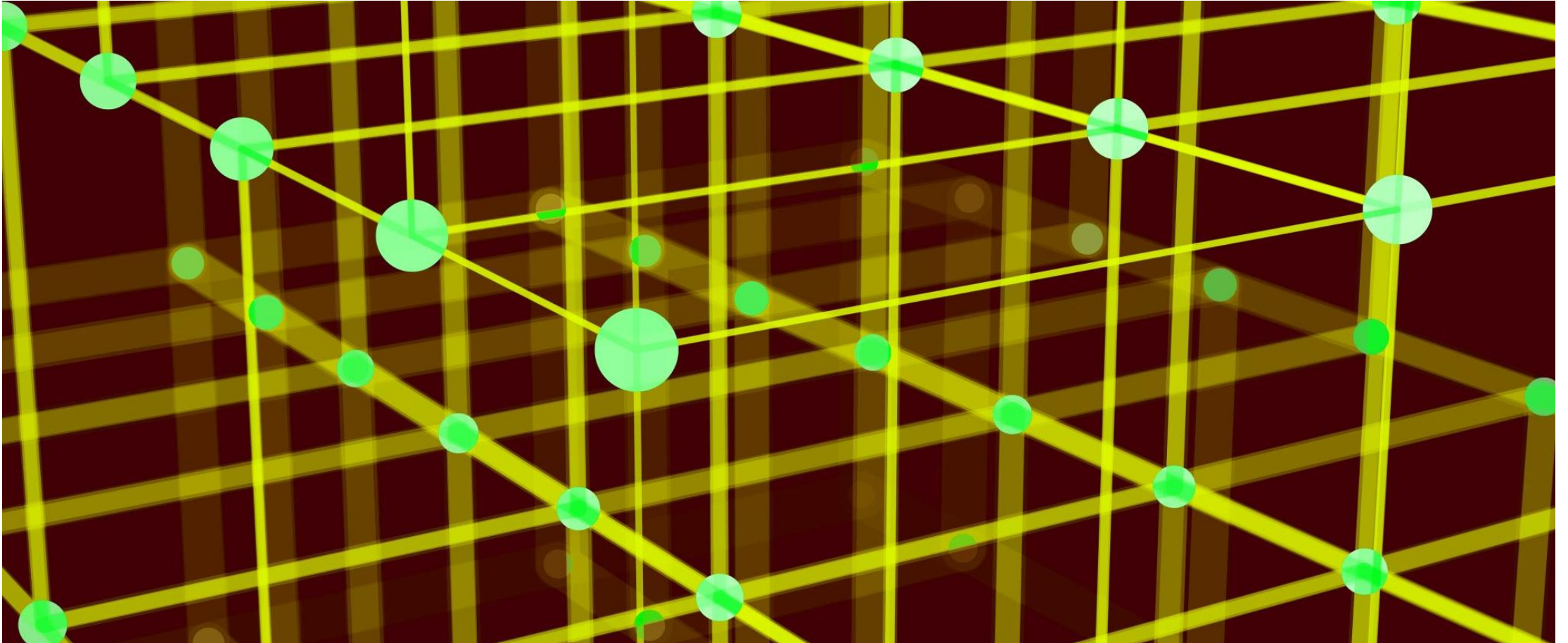


Inner life of a cell



<https://www.youtube.com/watch?v=yKW4F0Nu-UY>

Spatial SSA: the Next Subvolume Method

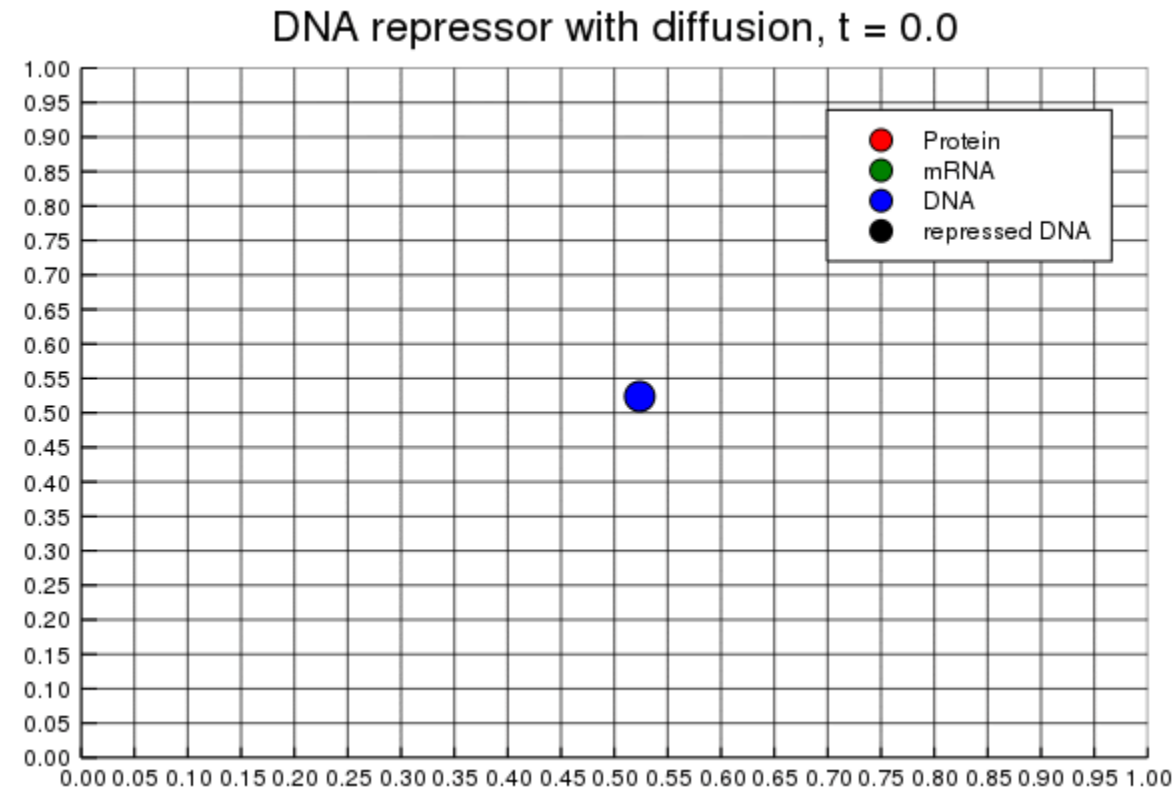
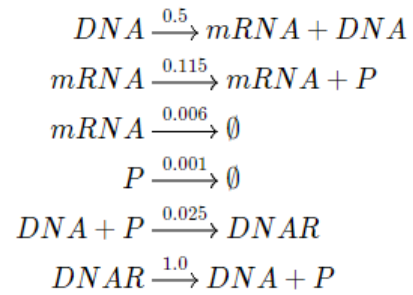




The main idea of NSM

- Instead of considering a single reaction volume, we **partition the space into multiple subvolumes**
 - Usually cubes of **side length ℓ**
 - The idea is that the molecules are **homogeneously distributed *inside* each subvolume**
 - We can perform a **SSA simulation** in each subvolume!
- One complication of this approach is that **molecules can diffuse** (i.e., move)
 - Instead of just calculating reactions, we must also take into account the **diffusion**
 - The diffusion of a molecule consists in **removing that molecule from a subvolume and adding that molecule into another adjacent subvolume**
- This is **NSM** in a nutshell, a simulator for **reaction-diffusion systems**
 - See [\[Elf and Ehrenberg, Systems Biology 2004\]](#)

Example of 2D NSM simulation

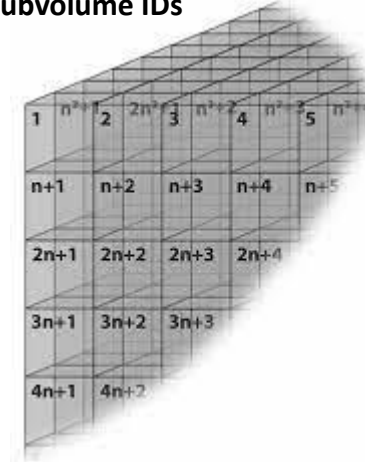


Connectivity matrix

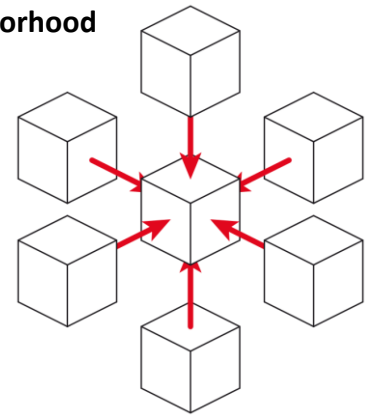


- In order to know **where molecules can go** we must describe the **topology** of the system, that is, the **neighbors of each subvolume**
- We can effectively implement that by means of a **Connectivity Matrix (CM)**
 - **Each subvolume** has a numeric ID
 - **Each row** in the CM corresponds to **one subvolume**
 - Assuming a Von Neumann neighborhood, each subvolume *can* have **up to 6 neighbors**
 - Thus, the CM has **6 columns**: one for each neighbor
 - We can choose in which subvolume the molecule will be diffused by **randomly picking one of the neighbors**
 - We can use the ID of the current subvolume to keep the molecule in the same position

Subvolume IDs



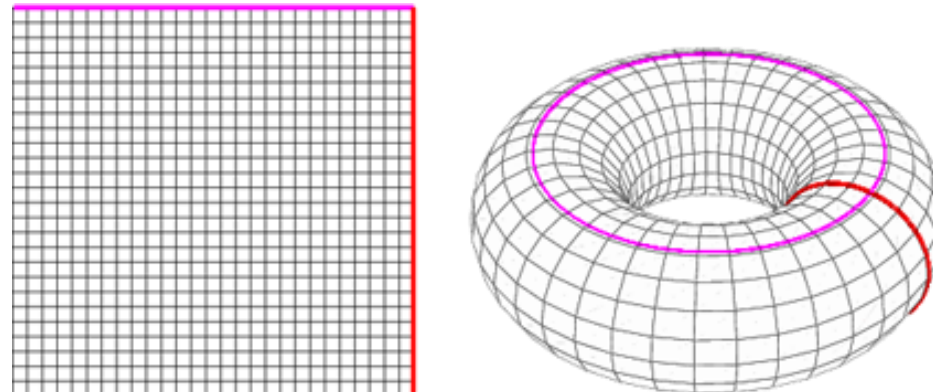
Von Neumann's neighborhood



Boundary conditions



- **Boundary conditions** specify what happens in the most
 - They can be specified by assigning the **appropriate neighbors** in the CM
 - Let's make an example; assume a flat reaction space
 - If the "right" neighborhood of the rightmost subvolumes (see the red line) are the leftmost subvolumes and the "upper" neighborhood of the upper subvolumes (see magenta line) are the bottom subvolumes, then we got **toroidal periodic boundary conditions**
 - That is, the molecules are **moving across a 3D toroid** (see figure)
 - Else, we have **"closed" boundary** conditions



Configuration



- What is the **state of the system** now?

Configuration



- What is the **state of the system** now?
- It is no longer just the number of molecules for each chemical species
- We now must keep track of the **number of molecules *in each subvolume***
- It can be implemented efficiently with a **configuration matrix**
 - One row for each subvolume
 - One column for each chemical species



Diffusion parameters

- The **parameters space** is extended as well
 - We still need some kinetic (**stochastic**) **parameters** for the reactions
 - But we also need **diffusion parameters**, to calculate the probability of **moving molecules** from one subvolume to a neighbor
- Please note that **even the arrival of a single molecule** (due to diffusion) into a subvolume can **radically change the propensities** and τ values
 - For instance, because such molecule appears as reactant for multiple reactions



Rate matrix [1/2]

- Given the **propensities** of reactions, we must **calculate their sum** (generally indicated as a_0) to determine the next reaction and waiting time τ
 - In this case, we will have a **sum for each compartment i** , named r_i
- Similarly, we will calculate the sum of **diffusion rates for each subvolume** using the following equation:

$$s_i = n_i \sum_{j=1}^M d_j X_j^i$$

where $d_j = D_j/\ell^2$ is the **diffusion rate constant** for species j , with $j = 1, \dots, M$, while n_i is the number of **directions** in which the molecule can diffuse



Rate matrix [2/2]

- The **rate matrix** is a table that contains the information about r_i and s_i values for all compartments
 - It also stores the **values** $r_i + s_i$ for all compartments
- The idea is that now we can **generate a random number** ρ (with uniform distribution in $[0,1]$) and **determine the first event time** in the subvolume (either reaction or diffusion) using Gillespie's approach:
 - This is basically a generalization of SSA
 - With a similar approach, we can also **generalize the choice of the reaction or diffusion event**



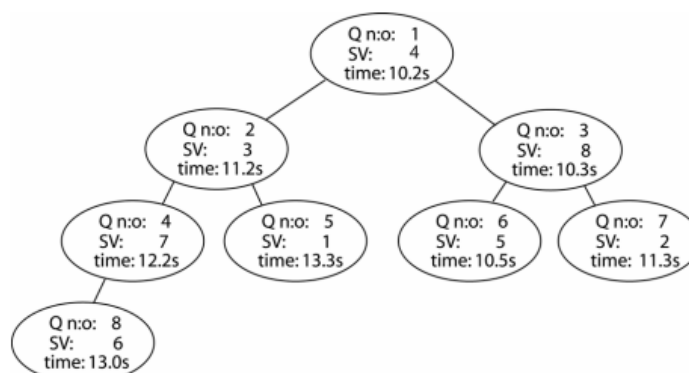
First Reaction Method

- You remember that, in the FRM, we begin by calculating the τ for **all reactions** and then deterministically **chose the reaction with the smallest τ value**
 - We must do **something similar here**: we do not want a different time in different subvolumes!
 - **The event** (reaction or diffusion) **with the smallest τ must be the first to happen**
- However, if we have M possible chemical reactions and D possible diffusion events, across N subvolumes, it means to recalculate $M \times D \times N$ values for τ
 - If we divide the 3D volume into a grid with 10 subvolumes per side, it is already $N = 10^3$
 - The fastest algorithms for sorting lists of real values have complexity $\mathcal{O}(n \cdot \log(n))$
 - **Way too many calculations**
- Remember that Gibson and Bruck used some tricks to reduce calculations: the dependency graph and the **priority queue** – we will do something similar for NSM

Summary of NSM's data structures

i	$n1$	$n2$	$n3$	$n4$	$n5$	$n6$	#A	#B	#C	$r_i [s^{-1}]$	$s_i [s^{-1}]$	$r_i + s_i [s^{-1}]$	Q
1	2	1	3	1	5	1	10	2	0	2.2	10	12.2	5
2	2	1	4	2	6	2	9	1	3	4.2	11.3	15.5	7
3	4	3	3	1	7	3	5	0	2	2.3	5.4	7.3	2
4	4	3	4	2	8	4	7	1	1	1.4	6.4	7.8	1
5	6	5	7	5	5	1	4	0	2	0.4	4.3	4.7	6
6	6	5	8	6	6	2	7	1	3	0.5	10.3	10.8	9
7	8	7	7	5	7	3	8	2	4	1.0	13.3	14.3	4
8	8	7	8	6	8	4	5	0	2	5.3	5.4	10.7	3

Connectivity matrix
Configuration
Rate matrix
Q-array



Event Queue

Position in Queue (Q)	Subvolume (SV)	τ_i (s)
1	4	10.2
2	3	11.2
3	8	10.3
4	7	12.2
5	1	13.3
6	5	10.5
7	2	11.3
8	6	13.0

Fig. M2. Data structures The structures within solid borders are arrays used in the algorithm. The *connectivity matrix* ($N \times 6$) stores the neighboring subvolumes' indices ($n1$ - $n6$) for each subvolume (*rows*). This defines the geometry and boundary conditions for the system. The *configuration matrix* ($N \times M$) stores the present number of molecules of each species in each cell. The *rate matrix* ($N \times 3$) stores the sum of reaction rate constants (r) and the sum of diffusion rate constants (d). The Q array keeps a reference to the subvolume's position in the *event queue*. In the *event queue* the subvolumes are ordered such that the one with the first scheduled event time (t) is at the top and each branch is sorted with increasing event times.

NSM algorithm

- If you check the pseudo-code carefully you realize that it is **basically SSA**
- It is extended to support a **new class of events** (diffusions)
- It exploits complex data structures to prevent useless and **time-consuming calculations**

The algorithm

Initialization

1. Generate a *connectivity matrix* (Fig. M2) that describes the geometry of the system.
2. Distribute the initial numbers of molecules between the subvolumes and store in the *configuration matrix*. This can be done randomly or according to any initial distribution.
3. Calculate the sum of reaction rates r_i for each subvolume i and store in the *rate matrix*. The reaction rates are calculated for the size Δ of the subvolume, as in the reaction-diffusion master equation.
4. Calculate the sum of diffusion rates for each subvolume, $s_i = n_i \sum_{j=1}^M d_j X_j^i$, where $d_j = D_j / \ell^2$ is the diffusion rate constant for species j . X_j^i is the number of molecules of species j in subvolume i . M is the number of species. n_i is the number of directions in which the molecules can diffuse. Store s_i in the *rate matrix*.
5. For each subvolume i : (a.) sum $r_i + s_i$, (b.) generate a random number, *rand*, uniformly distributed between 0 and 1 and (c.) calculate the first event time for each subvolume as $\tau_i = -\ln(\text{rand}) / (r_i + s_i)$.
6. Make an initial ordering of the subvolumes according to their next event times. The subvolumes are kept sorted in a binary tree (an *event queue*, see below) so that the subvolume for which the event occurs first is on the top and all branches have increasing event times.

Iterations

7. Assume that λ is the subvolume in which the next event occurs at time $t = \tau_\lambda$ according to the top element of the event queue. Generate a random number *rand* uniformly distributed between 0 and 1, choose a chemical reaction if $\text{rand} < r_\lambda / (r_\lambda + s_\lambda)$ and otherwise a diffusion jump.
8. Reaction event:
 - a. Rescale *rand* from (7.) linearly to [0,1] to determine which reaction occurred as in the direct method.
 - b. Update the state of the subvolume λ in the configuration matrix according to the state changes by reaction.
 - c. Recalculate the sum $r_\lambda + s_\lambda$ for the subvolume λ , generate a new random number and calculate the time of the next event as $t_\lambda = -\ln(\text{rand}) / (r_\lambda + s_\lambda) + t$.
 - d. Insert the active subvolume's new event time in the event queue and order the queue (see below).
9. Diffusion event:
 - a. Rescale *rand* from (7.) linearly to [0,1] to determine which type of molecule that diffused away. The diffusion intensities are given by the numbers of the different types of molecules weighted by their diffusion rate constants.
 - b. The direction of the diffusion event is chosen by randomly selecting a column in the connectivity matrix. This can be done by rescaling the random number used in 9a (again).
 - c. Update the states of both subvolume λ and its neighbor, γ , that got an additional molecule.
 - d. Recalculate the sums $r_\lambda + s_\lambda$ and $r_\gamma + s_\gamma$, sample the time to the next event in the subvolumes as in 8c.
 - e. Insert the subvolumes' new event times in the event queue and order the queue (see below).
10. Return to 7. for the next iteration.



A few subtleties

- How to choose the **size of the subvolumes**?
 - Well, at least **big enough to contain proteins and complexes!**
 - **Not too big**, because that would violate the **well-stirred assumption** within the subvolumes
 - How do we handle the situation of **overcrowding**?
 - The limit $\ell \rightarrow 0$ is **not physically attainable** AND **molecules never react!**
(see [\[Isaacson, SIAM J Appl Math 2009\]](#))
- How to choose the **diffusion parameters**?
 - Well, at least **slower than the diffusion of water**
 - We can **estimate** them (just like kinetic parameters – more about this in a future lecture)

Space subdivisions

- It is not mandatory to subdivide the space into **cubic** subvolumes
- However, we **diffusion mechanisms and parameters** change according to the geometry
- Also, defining a topology can be challenging

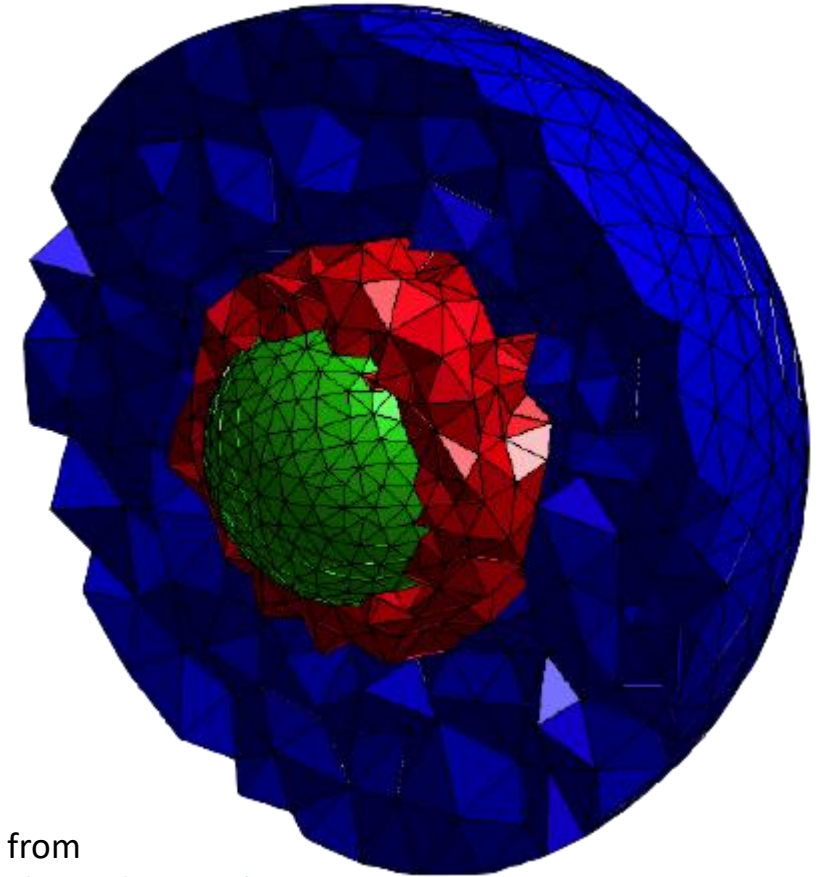
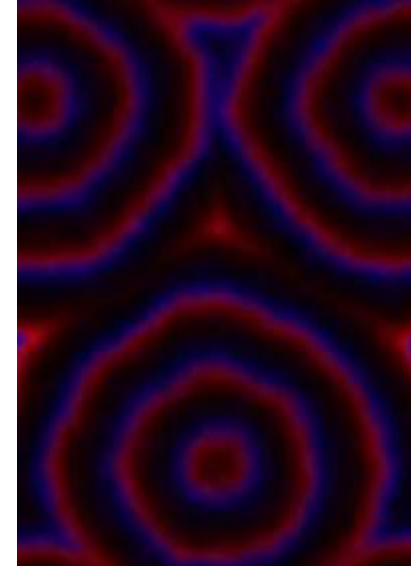


Figure from
[[Hellander and Lötstedt, SIAM J Multiscale Model Simulat 2012](#)]

Stochastic spatial simulation of Lotka-Volterra



- <https://www.youtube.com/shorts/UsYNGSUIS6o>
- Courtesy of Prof. Darren Wilkinson





Summary for the second part

- Some complex systems are inhomogeneous, that is, the "well-stirred" assumption is not valid because some items are localized in space
- Under these circumstances, conventional simulators based on counting the number of molecules can yield misleading results
- One strategy to keep spatiality into account is to subdivide the reaction volume into multiple subvolumes, and use a normal simulator (e.g., SSA) in each subvolume
- However, if molecules can diffuse, we need to keep that phenomenon into account
- Due to the nature of SSA, we also need strategies to synchronize the simulation across multiple subvolumes
- The Next Subvolume Method was designed to achieve that efficiently, by using several dedicated data structures

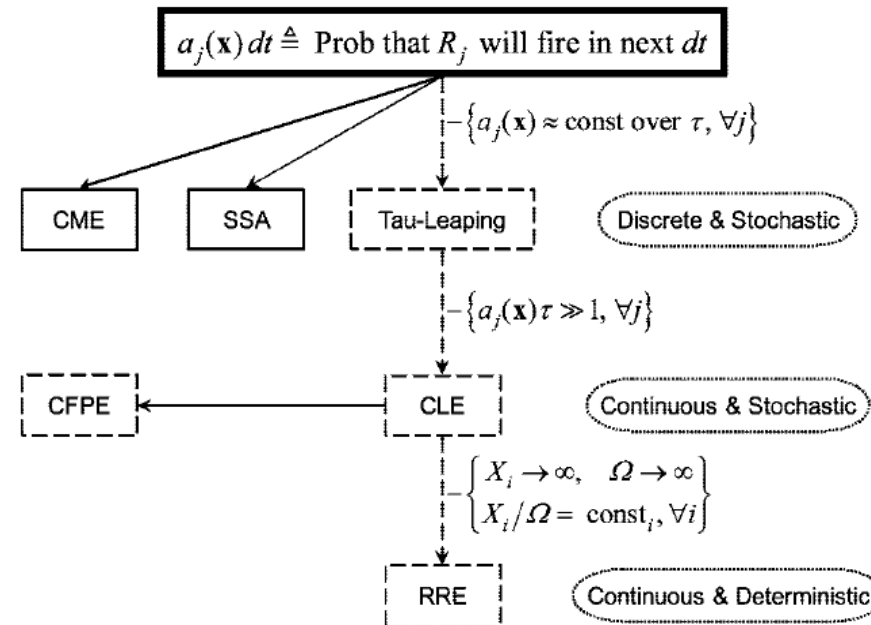
Chemical Langevin Equation



The chemical Langevin equation

J. Chem. Phys. 113, 297 (2000); <https://doi.org/10.1063/1.481811>

Daniel T. Gillespie



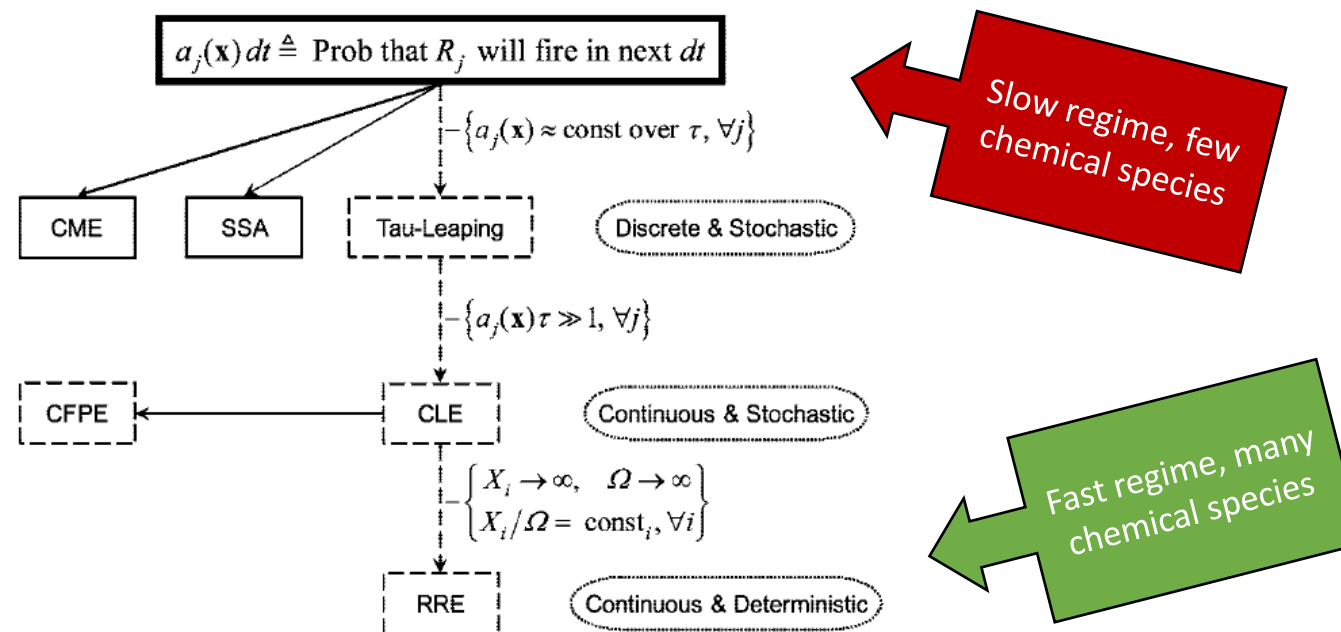
Different scales



The chemical Langevin equation

J. Chem. Phys. 113, 297 (2000); <https://doi.org/10.1063/1.481811>

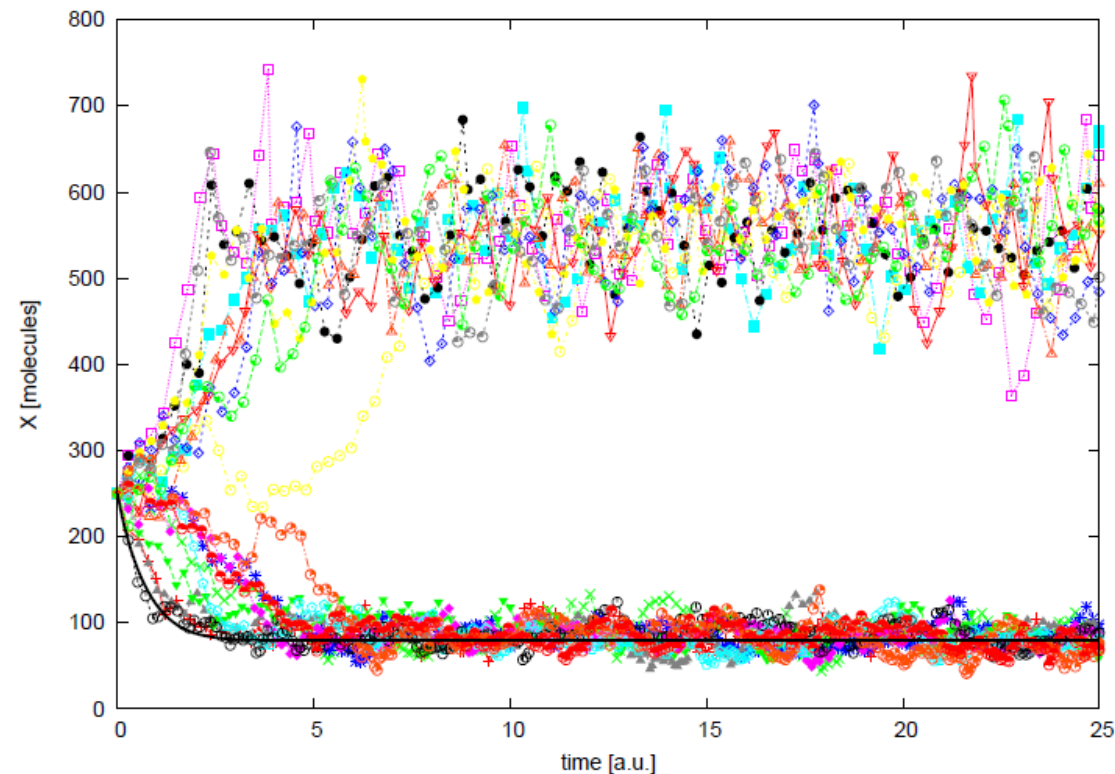
Daniel T. Gillespie



Preserving emergent phenomena

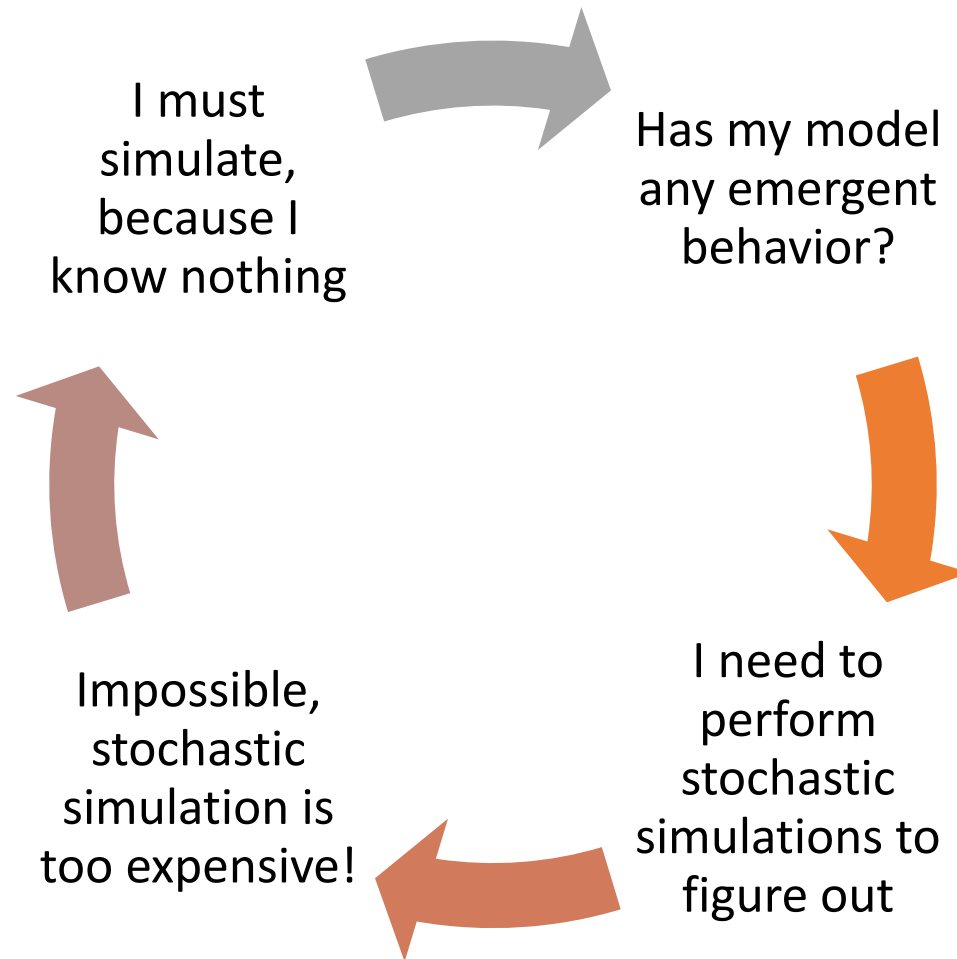


- Stochasticity is often necessary to fully reproduce an observed behavior



Black line =
ODE!

A circular paradox



Hybrid stochastic simulation



The Journal of Chemical Physics



SUBMIT

HOME

BROWSE

INFO

FOR AUTHORS

COLLECTIONS

ACCEPTED MANUSCRIPTS



SIC

[Home](#) > [The Journal of Chemical Physics](#) > [Volume 117, Issue 15](#) > [10.1063/1.1505860](https://doi.org/10.1063/1.1505860)

 Full • Submitted: 03 June 2002 • Accepted: 17 July 2002 • Published Online: 30 September 2002

Approximate simulation of coupled fast and slow reactions for stochastic chemical kinetics

J. Chem. Phys. **117**, 6959 (2002); <https://doi.org/10.1063/1.1505860>

Eric L. Haseltine *and* James B. Rawlings



Coupled fast/slow reactions

- Computational costs of Gillespie-like algorithms **increase with "fast" events/reactions**
 - Fast reactions are often due to a **high concentration of reactants**
 - **Not all reactions in the system are fast**, some happen with a **low probability**
- [Haseltine and Rawlings, J Chem Phys 2002] propose to **partition the system** into subsets of "fast" and "slow" reactions
 - Fast reactions can be **simulated using ODEs or Langevin equations (SDE)**
 - Slow reactions are simulated with **SSA**
 - They recommended to maintain "at least a **two orders of magnitude difference** between the partitioned reaction probabilities"

Up- and down-sides with multi-scale



- Pro: **you do not have to choose** a modeling/simulation approach!
- Con: we are introducing an **approximation** that might **change the distributions** of our variables
 - See Figure from [\[Liu et al., J Chem Phys 2012\]](#)
- Con: SSA and ODEs proceed with **different time-steps!**
 - We need some kind of **synchronization** policy

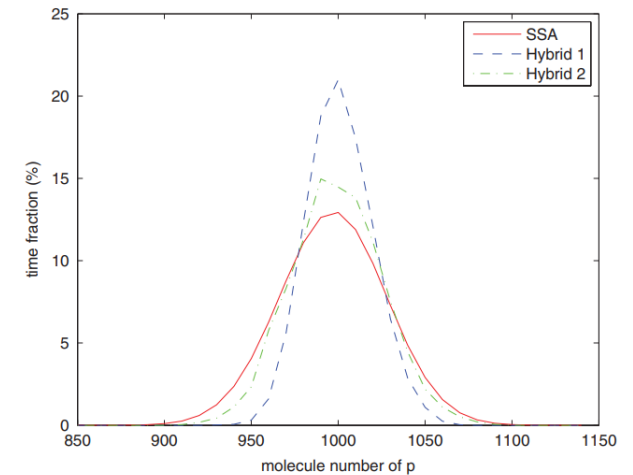
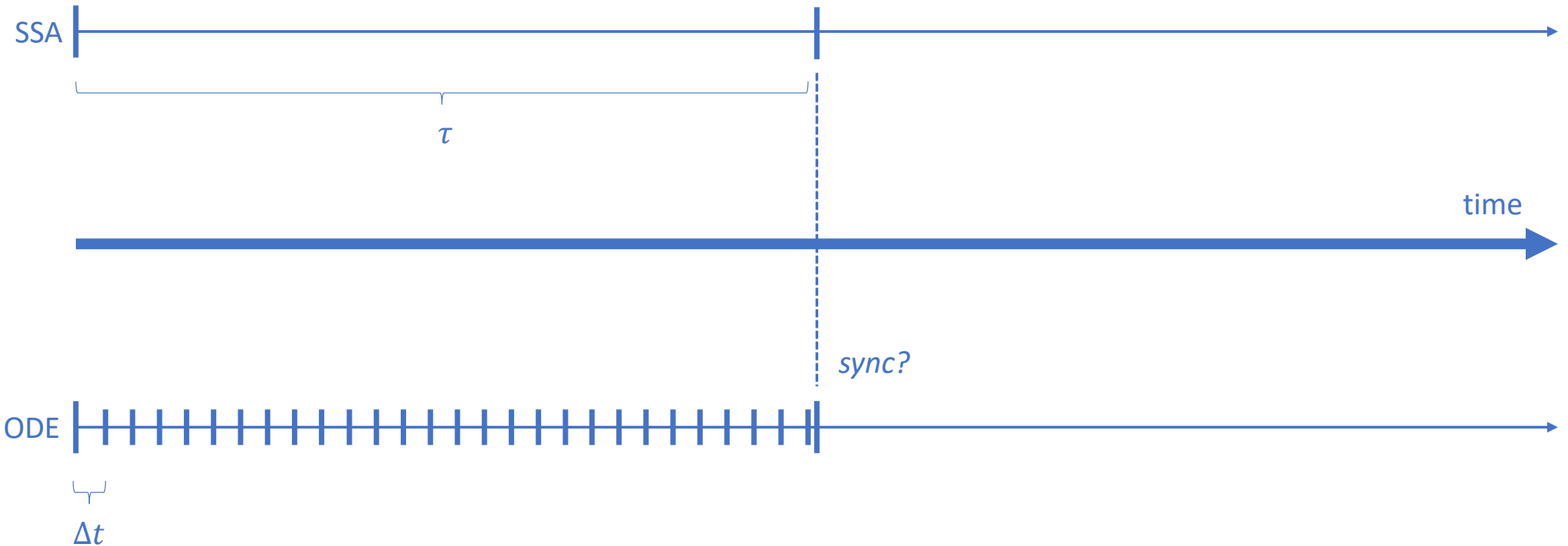
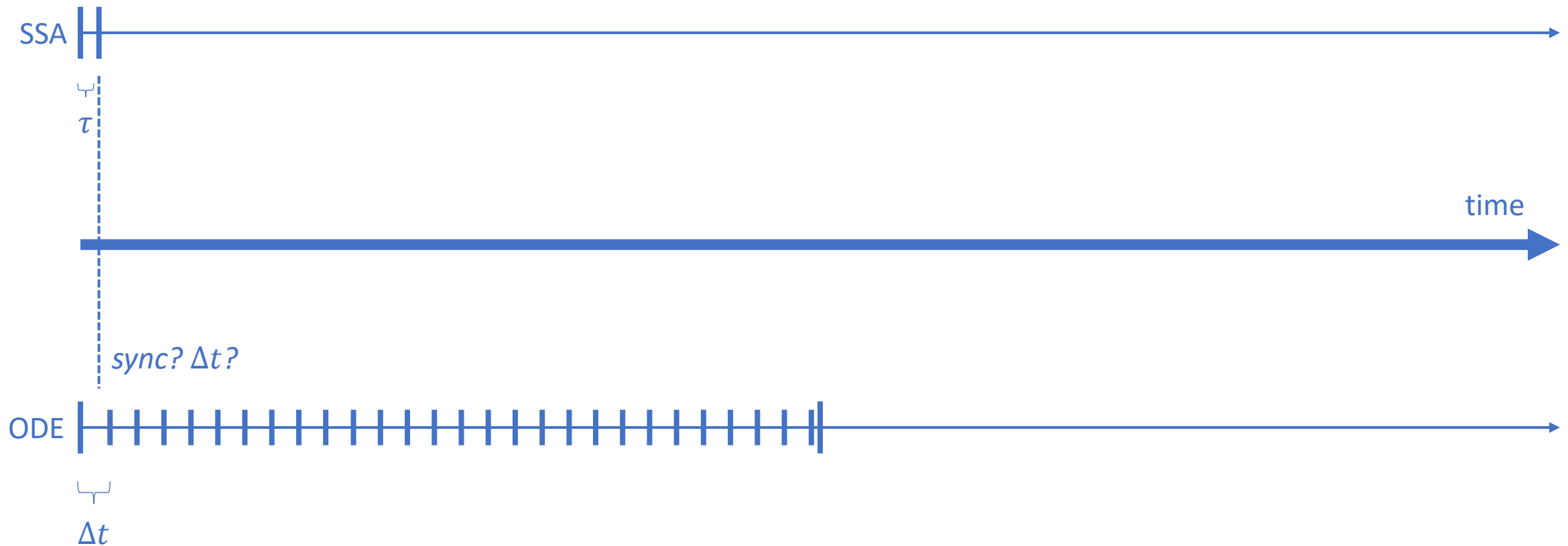


FIG. 10. Distribution of p population for the steady-state model.

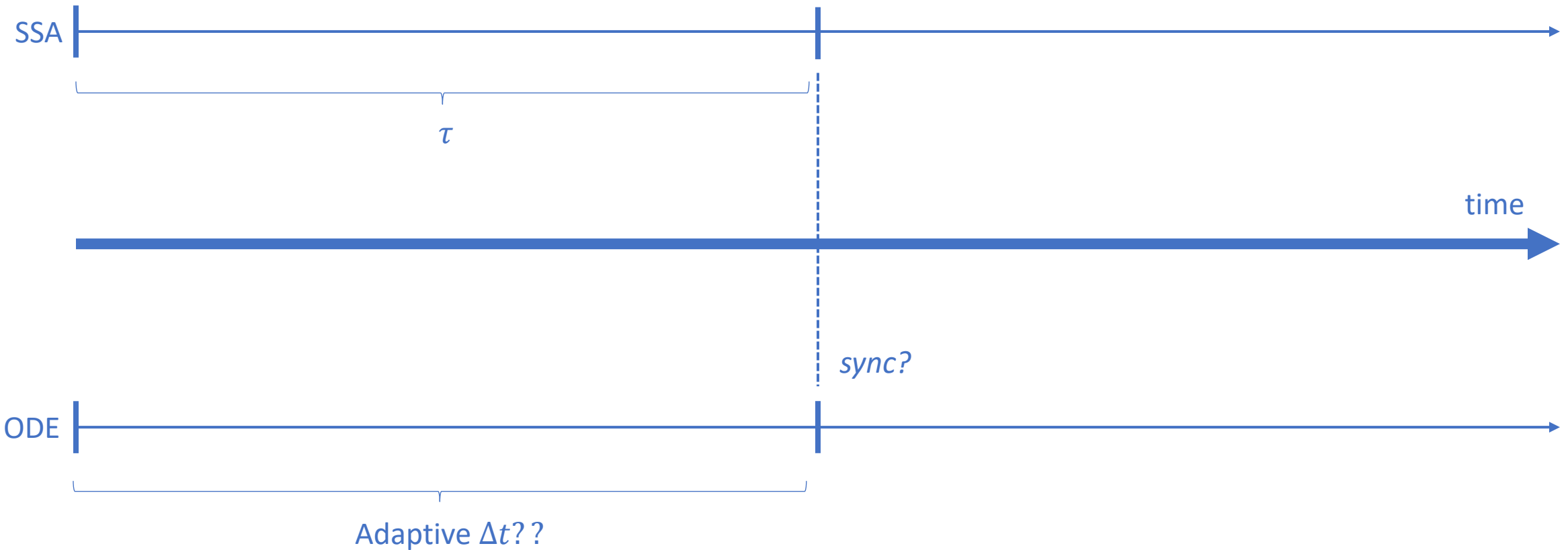
Why synchronization?



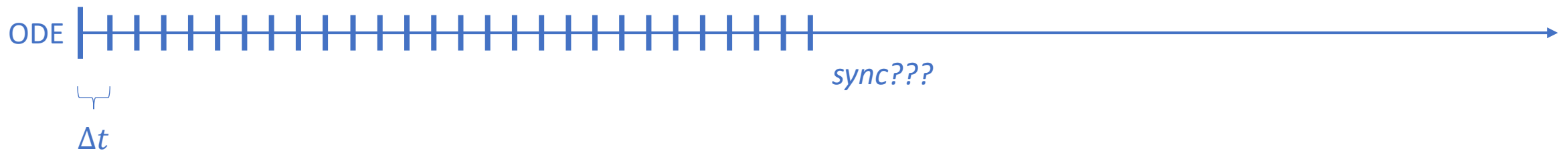
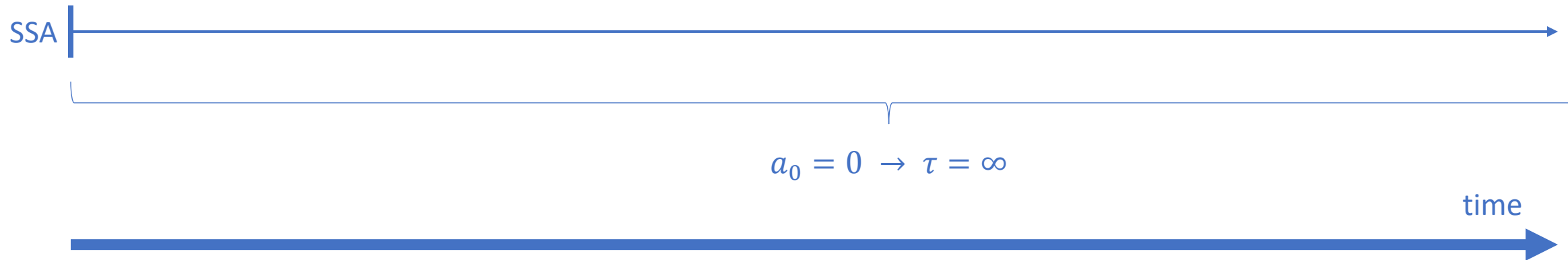
Why synchronization?



Why synchronization?



Why synchronization?



Multi-scale simulation



Progress in Biophysics & Molecular Biology 85 (2004) 217–234

Progress in
**Biophysics
& Molecular
Biology**

www.elsevier.com/locate/pbiomolbio

A multi-scaled approach for simulating chemical reaction systems

Kevin Burrage*, Tianhai Tian, Pamela Burrage

*Department of Mathematics, Advanced Computational Modelling Centre, University of Queensland,
Brisbane QLD4072, Australia*

Burrage's multi-scale method



- Subdivision into **three regimes**:
 - **Slow regime**: simulated with SSA
 - **Intermediate regime**: simulated with τ -leaping
 - **Fast regime**: simulated with CLE (explicit Euler-Maruyama)
- How to partition the reactions in these three regimes?



Burrage's multi-scale method

- Subdivision into **three regimes**:
 - **Slow regime**: simulated with SSA
 - **Intermediate regime**: simulated with τ -leaping
 - **Fast regime**: simulated with CLE (explicit Euler-Maruyama)
- How to partition the reactions in these three regimes?
 - In general, it seems a good idea to have a **dynamic partitioning** adapting to quickly **changing conditions**
 - [Burrage et al., Prog Biophys Mol Biol 2004] attempted a **rigorous mechanism** for regime classification



Burrage *et al.* regimes classification

Table 1
Regime classification

[Burrage et al., Prog Biophys Mol Biol 2004]

# Molecules	Propensity function	Classification
Large	Large	Fast
Moderate	Large	Intermediate
Large	Moderate	Intermediate
Moderate	Moderate	Intermediate
Large	Small	Slow
Small	Large	Slow
Moderate	Small	Slow
Small	Moderate	Slow
Small	Small	Slow

$X_i \leq 100 \Rightarrow \text{small},$

$X_i \in [101, 1000] \Rightarrow \text{moderate},$

$X_i > 1000 \Rightarrow \text{large}$

$a_i(X) \leq 5 \Rightarrow \text{small},$

$a_i(X) \in (5, 100] \Rightarrow \text{moderate},$

$a_i(X) > 100 \Rightarrow \text{large}.$



Burrage's multi-scale method

- **Three regimes:**
 - **Slow regime:** simulated with SSA
 - **Intermediate regime:** simulated with τ -leaping
 - **Fast regime:** simulated with CLE (explicit Euler-Maruyama)
- Note that both the slow and intermediate regimes proceed with **irregular time steps** τ_S and τ_I , respectively
- This is not the case of Euler-Maruyama's step τ_F !

Calculating the time steps [1/3]

- The slow regime uses Gillespie's calculations for the time step τ_S

$$\tau_S = \frac{1}{a_{slow}^{(0)}(\mathbf{X}(t))} \ln \left(\frac{1}{\rho} \right)$$

- where ρ is a random number in $[0,1]$ and
- a_{slow}^0 is the sum of the propensities of reactions belonging to the slow regime

Calculating the time steps [2/3]

$$\bullet \tau_I = \min_{j \in Int} \left\{ \frac{a_j(\mathbf{X}(t)+v) - a_j(\mathbf{X}(t))}{a_{mod}^{(0)}} \right\} < \varepsilon$$

- Where ε is a user-selected parameter bounding the relative change of propensities during the leap
- Int is the set of reactions in the intermediate regime
- mod is the set of propensities classified as moderate

Calculating the time steps [3/3]



- If $\tau_I < \tau_S$ then the algorithm only performs τ -leaping until the simulation reaches $t + \tau_S$
- If $\tau_I \geq \tau_S$ then $\tau_I = \tau_S$
- To guarantee stability, $\tau_F \ll \tau_S$



Additional issues with hybrid systems...

- In the fast regimes we use ODEs and CLEs
 - → The variables of the system are **continuous** (e.g., concentrations)
- In the slow regimes we use SSA and τ -leaping
 - → The variables are **discrete** (e.g., protein counts)
- **The two regimes communicate and share an overall state** of the system, but we must **round the concentrations** to a discrete number of particles, and vice versa!
 - This process might introduce some (negligible?) errors

Dealing with stiffness...



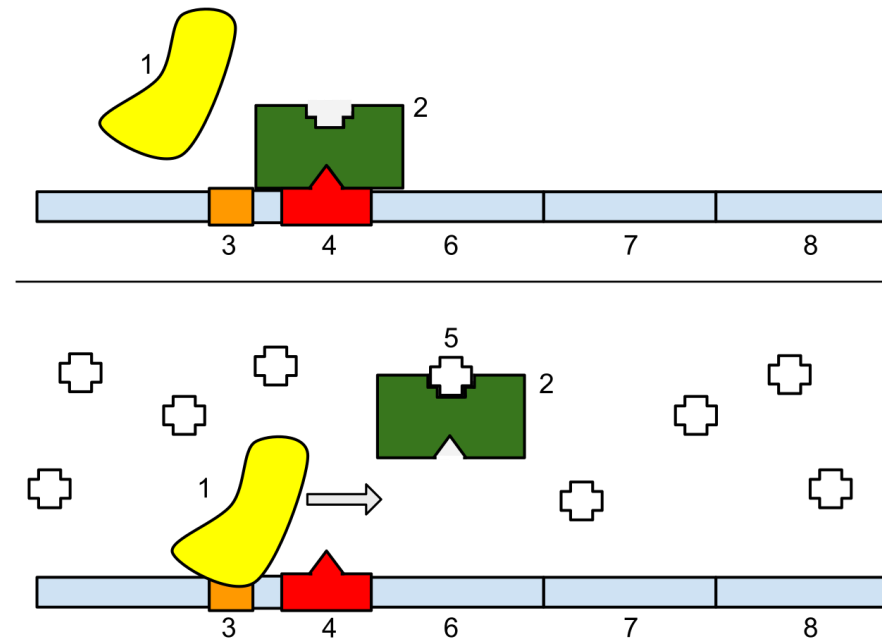
- Haseltine and Rawlings use an explicit method, namely the Euler–Maruyama method, for simulating the CLE. However, since the propensity functions in the CLE are large, the SDE is stiff (in the sense of widely varying Lyapunov exponents) and thus some consideration could be given to using semi-implicit or fully-implicit methods for this component. This could come at some cost if the dimension of the fast subsystem is at least moderately large.

[Burrage et al., Prog Biophys Mol Biol 2004]

Example of Burrage's algorithm [1/2]



- Model: expression and activity of **LacZ and LacY lactose operons** in *E. coli*
 - François Jacob and Jacques Monod won the **Nobel prize** for their research on Lac operons
 - **Complex multi-scale model** with 22 reactions and 23 species [\[Kierzek, Bioinformatics 2002\]](#)



Example of Burrage's algorithm [2/2]



- When the system reaches the **steady-state** there are **three well-separated simultaneous regimes**:
 - The **amounts** of lactose, lacZ and RbsLacY are 30000, 300 and 2, respectively
 - The **propensities** of reactions 5, 17 and 20 are 0.003, 0.1 and 1000, respectively

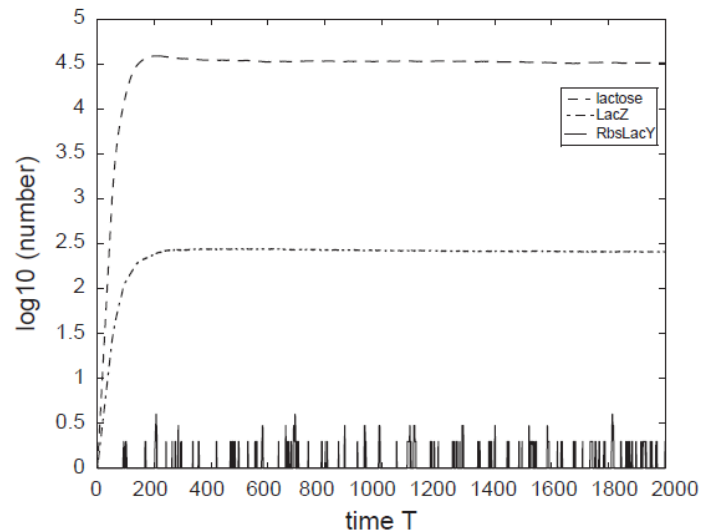


Fig. 1. Log of number of molecules vs. time.

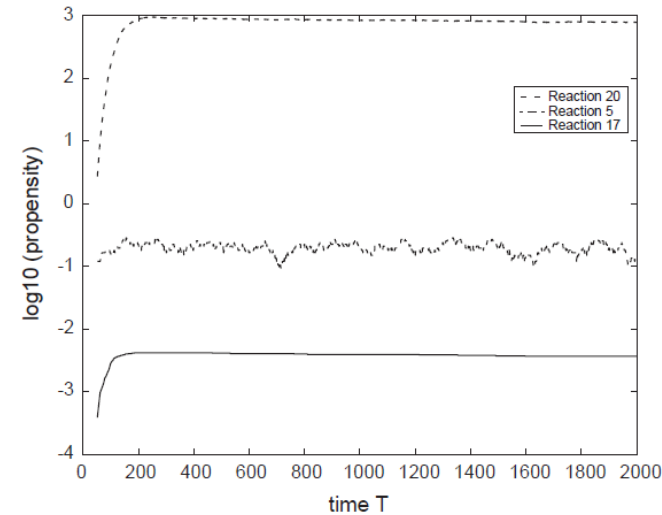


Fig. 2. Log of propensity functions vs. time.

The Lac operon model



A full list of reactions and rates

	Reaction	Rate constant
1	$\text{PLac} + \text{RNAP} \rightarrow \text{PLacRNAP}$	0.17
2	$\text{PLacRNAP} \rightarrow \text{PLac} + \text{RNAP}$	10
3	$\text{PLacRNAP} \rightarrow \text{TrLacZ1}$	1
4	$\text{TrLacZ1} \rightarrow \text{RbsLacZ} + \text{PLac} + \text{TrLacZ2}$	1
5	$\text{TrLacZ2} \rightarrow \text{TrLacY1}$	0.015
6	$\text{TrLacY1} \rightarrow \text{RbsLacY} + \text{TrLacY2}$	1
7	$\text{TrLacY2} \rightarrow \text{RNAP}$	0.36
8	$\text{Ribosome} + \text{RbsLacZ} \rightarrow \text{RbsRibosomeLacZ}$	0.17
9	$\text{Ribosome} + \text{RbsLacY} \rightarrow \text{RbsRibosomeLacY}$	0.17
10	$\text{RbsRibosomeLacZ} \rightarrow \text{Ribosome} + \text{RbsLacZ}$	0.45
11	$\text{RbsRibosomeLacY} \rightarrow \text{Ribosome} + \text{RbsLacY}$	0.45
12	$\text{RbsRibosomeLacZ} \rightarrow \text{TrRbsLacZ} + \text{RbsLacZ}$	0.4
13	$\text{RbsRibosomeLacY} \rightarrow \text{TrRbsLacY} + \text{RbsLacY}$	0.4
14	$\text{TrRbsLacZ} \rightarrow \text{LacZ}$	0.015
15	$\text{TrRbsLacY} \rightarrow \text{LacY}$	0.036
16	$\text{LacZ} \rightarrow \text{dgrLacZ}$	6.42E-5
17	$\text{LacY} \rightarrow \text{dgrLacY}$	6.42E-5
18	$\text{RbsLacZ} \rightarrow \text{dgrRbsLacZ}$	0.3
19	$\text{RbsLacY} \rightarrow \text{dgrRbsLacY}$	0.3
20	$\text{LacZ} + \text{lactose} \rightarrow \text{LacZlactose}$	9.52E-5
21	$\text{LacZlactose} \rightarrow \text{product} + \text{LacZ}$	431
22	$\text{LacY} \rightarrow \text{lactose} + \text{LacY}$	14

Performances of Burrage's algorithm



- Simulation of the model with **SSA**
 - Running time **>7 hours**
 - 4.8e6 steps
- Simulation with Burrage's **multi-scale approach**:
 - Running time **5 hours**
 - 3.2e6 steps

Why dynamic partitioning?



The Journal of Chemical Physics



SUBM

HOME

BROWSE

INFO

FOR AUTHORS

COLLECTIONS

ACCEPTED MANUSCRIPTS



[Home](#) > [The Journal of Chemical Physics](#) > [Volume 122, Issue 5](#) > [10.1063/1.1835951](#)

 Full • Submitted: 06 May 2004 • Accepted: 28 October 2004 • Published Online: 14 January 2005

Accurate hybrid stochastic simulation of a system of coupled chemical or biochemical reactions

J. Chem. Phys. **122**, 054103 (2005); <https://doi.org/10.1063/1.1835951>

Howard Salis *and* Yiannis Kaznessis

Cyrcadian rythm in *D. melanogaster*



- Cyrcadian rythm: **internal biological clock**
- The model is a **pulse generator**
 - Two oscillating proteins (PER and TIM) dimerize and inhibit their own genes
 - Quick transition of regimes (fast to slow and vice versa): **any fixed partition would be wrong**

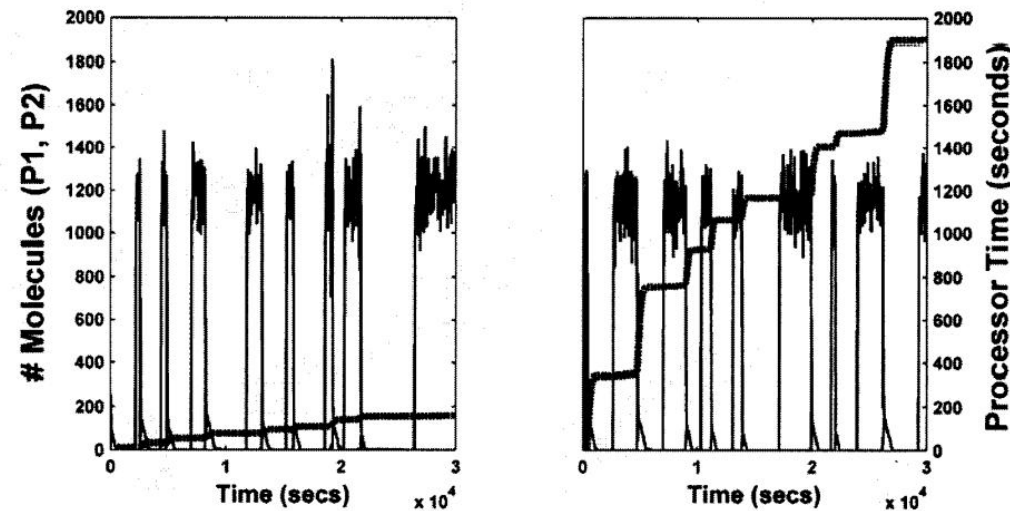


FIG. 5. The oscillatory dynamics of monomers P1 and P2 of the Pulse Generator reaction system (solid lines) are compared alongside their computational running times (dashed lines), using the Next Reaction hybrid method (left) and SSA (right). Note that the right Y-axis is the number of seconds the processor requires to simulate the system up to that point.

Modello di Harris (Partitioned leaping)



The Journal of Chemical Physics



HOME

BROWSE

INFO

FOR AUTHORS

COLLECTIONS

ACCEPTED MANUSCRIPTS

[Home](#) > [The Journal of Chemical Physics](#) > [Volume 125, Issue 14](#) > [10.1063/1.2354085](#)

 Full • Submitted: 30 January 2006 • Accepted: 16 August 2006 • Published Online: 11 October 2006

A “partitioned leaping” approach for multiscale modeling of chemical reaction dynamics

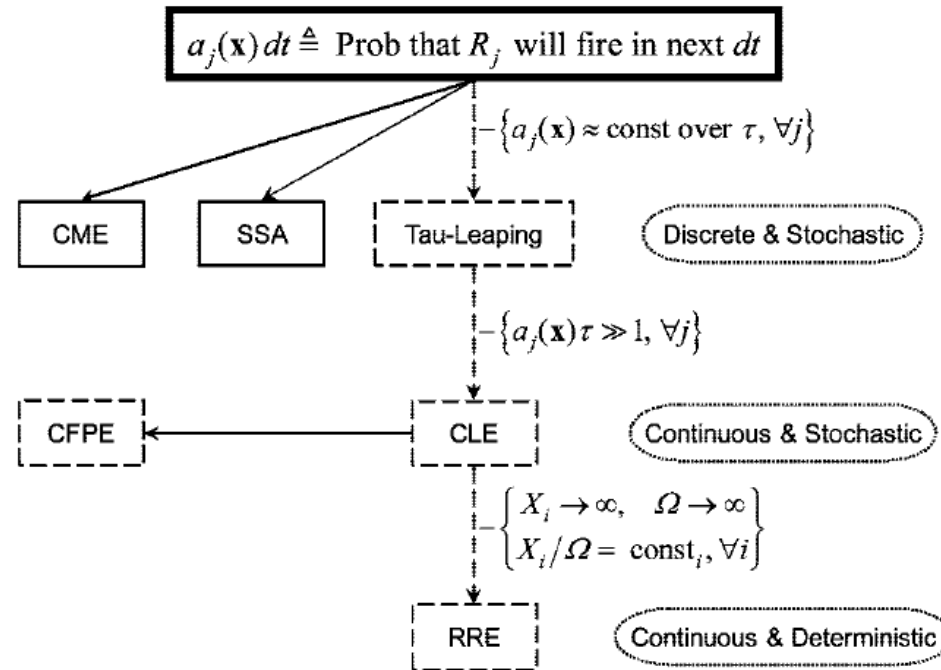
J. Chem. Phys. **125**, 144107 (2006); <https://doi.org/10.1063/1.2354085>

Leonard A. Harris^{a)} and Paulette Clancy^{b)}

Partitioned leaping

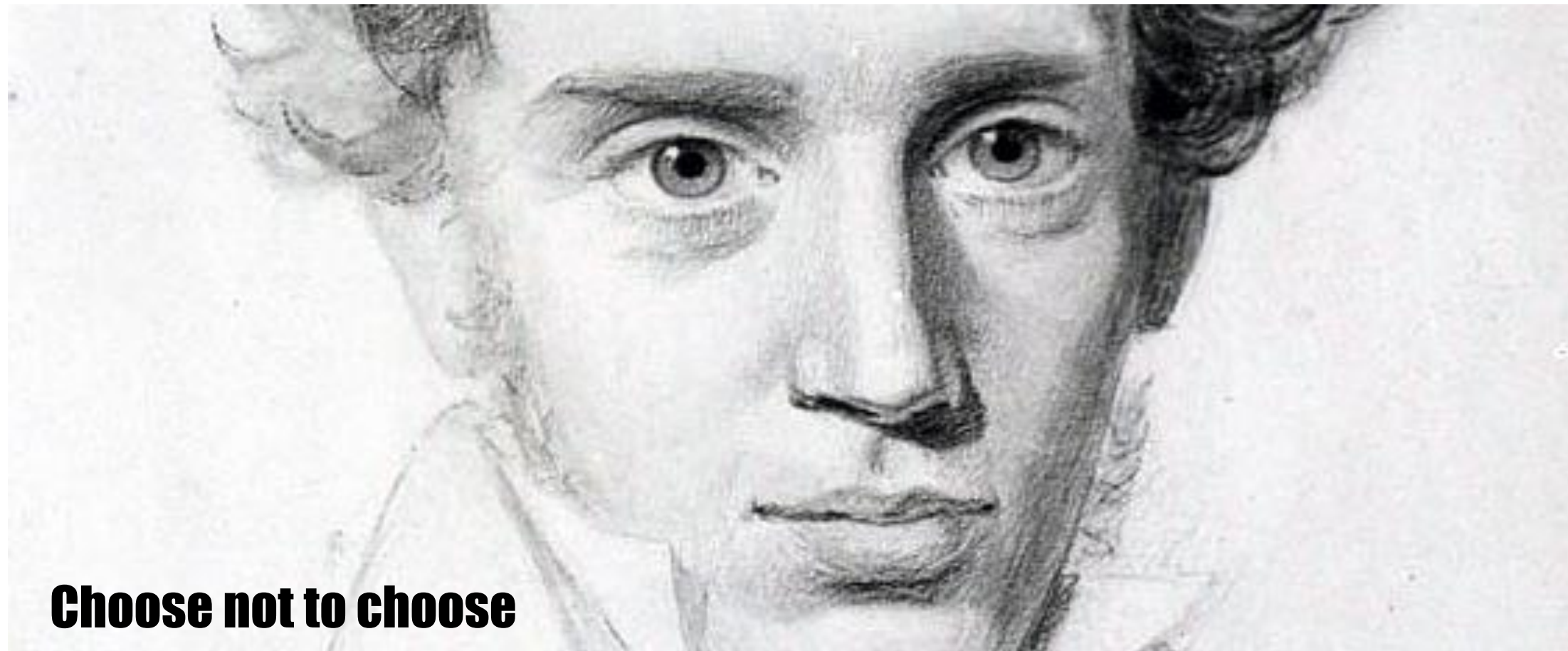


- Puts together **all regimes**: discrete stochastic (SSA, τ -leaping), continuous stochastic (CLE) and continuous deterministic (ODEs)



ral

ap



Choose not to choose

Partitioning according to propensity values



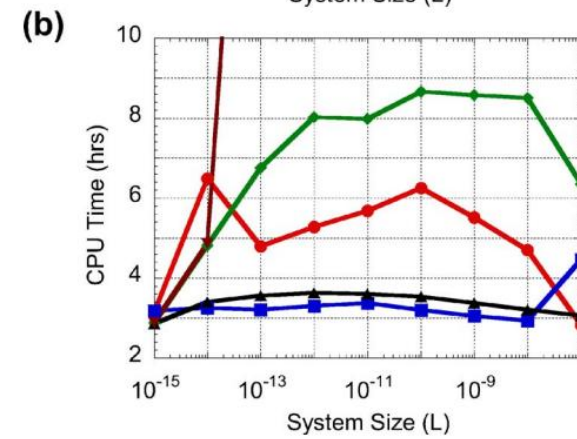
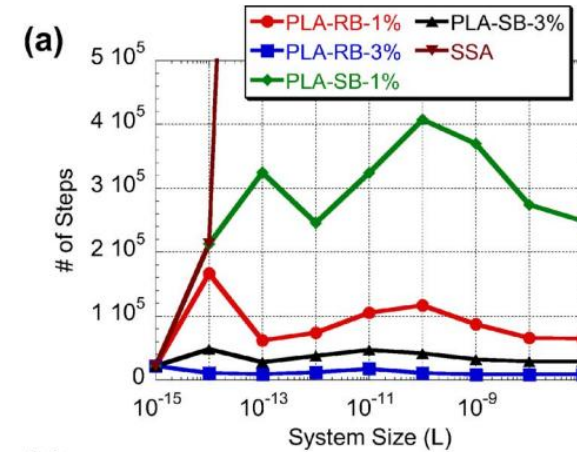
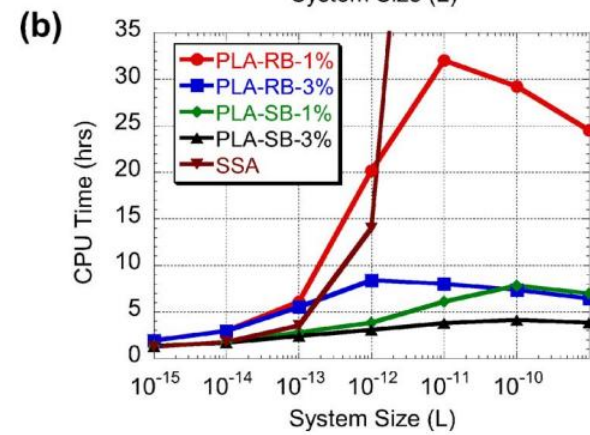
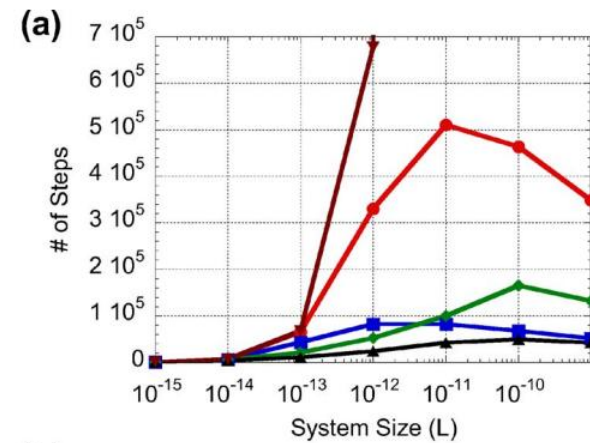
This procedure amounts to a theoretically justifiable partitioning scheme in which reactions are classified into four different categories based on their propensity values, the calculated time step τ , and the criteria identified by Gillespie^{16,37} for transitioning between the descriptions (5)–(7). The classifications are made as follows.

- If $a_\mu \tau \lesssim 1 \rightarrow$ *exact stochastic* (very slow).
- If $a_\mu \tau > 1$ but $\not\gg 1 \rightarrow$ *Poisson* (slow).
- If $a_\mu \tau \gg 1$ but $\sqrt{a_\mu} \tau \not\gg 1 \rightarrow$ *Langevin* (medium).
- If $\sqrt{a_\mu} \tau \gg 1 \rightarrow$ *deterministic* (fast).

Partitioned Leaping Algorithm (PLA)

- If $a_\mu\tau \leq 1 \rightarrow$ *exact stochastic* (very slow).
 - If $a_\mu\tau > 1$ but $\not\gg 1 \rightarrow$ *Poisson* (slow).
 - If $a_\mu\tau \gg 1$ but $\sqrt{a_\mu\tau} \not\gg 1 \rightarrow$ *Langevin* (medium).
 - If $\sqrt{a_\mu\tau} \gg 1 \rightarrow$ *deterministic* (fast).
- (1) Initialize (species populations, rate constants, define $\approx 1, \gg 1, \epsilon$, etc.).
 - (2) Determine the *initial* value of τ (see Sec. III B).
 - (3) Classify all reactions (not already classified as ES) using the criteria presented above.
 - (4) For all (newly classified) ES reactions, calculate tentative next-reaction times, τ_μ^{ES} , using Eqs. (3) and (4).
 - (5) If $\min\{\tau_\nu^{\text{ES}}\} \neq \tau$ and all reactions are ES, set $\tau = \min\{\tau_\nu^{\text{ES}}\}$.
 - (6) If $\min\{\tau_\nu^{\text{ES}}\} < \tau$, set $\tau = \min\{\tau_\nu^{\text{ES}}\}$ and return to step (3).
 - (7) Determine the set of reaction firings $\{k_\nu(\tau)\}$ using the appropriate formulas⁴³ and update the species populations.
 - (8) If any $X_i(t+\tau) < 0$, reverse all population updates, set $\tau = \tau/2$, and return to step (3). If not, advance the clock by τ and return to step (2) if stopping criterion is not met.

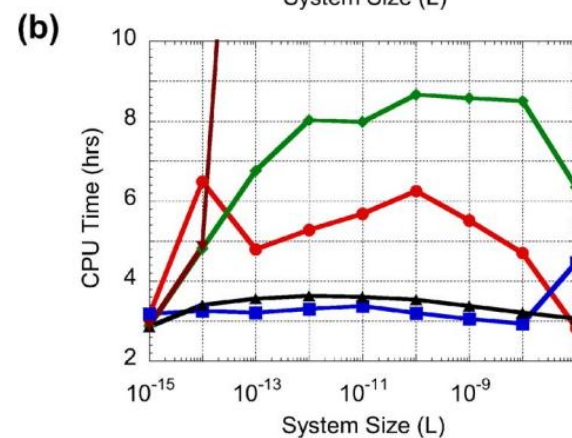
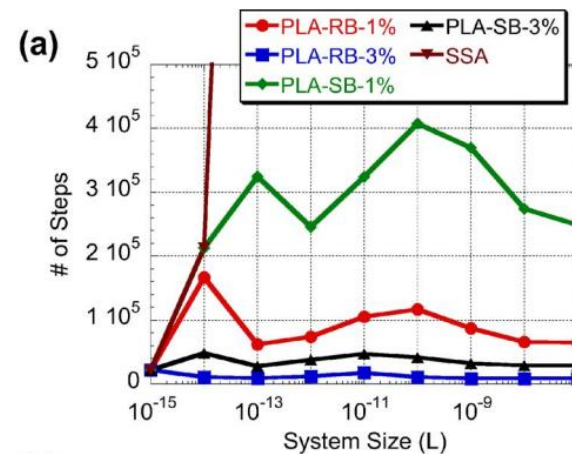
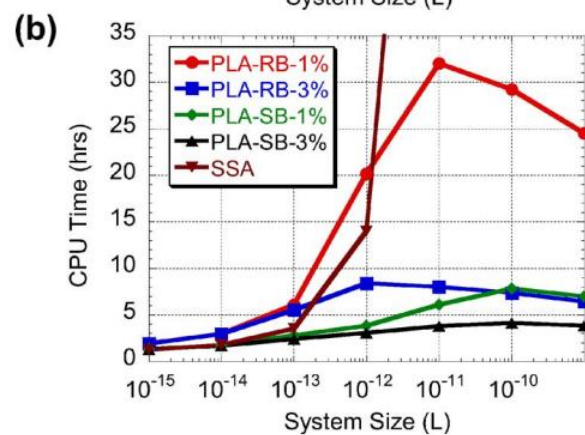
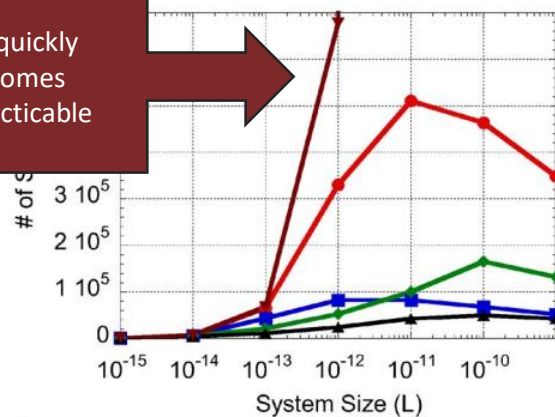
Performances of PLA



Performances of PLA



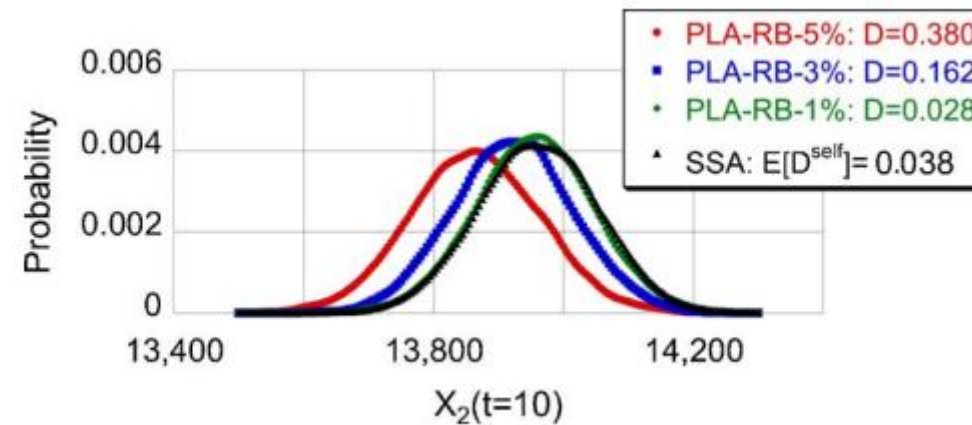
SSA quickly becomes impracticable



PLA with high error tolerance scales sublinearly

The inevitable approximation

- Simple dimerization/decay model: $S_1 \rightarrow \emptyset, S_1 + S_1 \leftrightarrow S_2, S_2 \rightarrow S_3$



Extensions: spatial PLA



The Journal of Chemical Physics



HOME

BROWSE

INFO

FOR AUTHORS

COLLECTIONS

ACCEPTED MANUSCRIPTS

[Home](#) > [The Journal of Chemical Physics](#) > [Volume 132, Issue 9](#) > [10.1063/1.3310808](#)



Full • Submitted: 04 September 2009 • Accepted: 16 January 2010 • Published Online: 01 March 2010

Accurate implementation of leaping in space: The spatial partitioned-leaping algorithm

J. Chem. Phys. **132**, 094101 (2010); <https://doi.org/10.1063/1.3310808>

Krishna A. Iyengar^{1, a)}, Leonard A. Harris^{2, b)}, and Paulette Clancy^{2, c)}

Leonard's explanation of S-PLA



A. Exact-stochastic methods

1. Well-mixed systems

Gillespie's SSA operates within a fully well-mixed system (i.e., $L = 1$).^{11,12} The approach determines *when* the next reaction will fire in the system and of *which* type it will be. Two mathematically equivalent approaches were presented for accomplishing this: the direct method (see Ref. 13 for details) and the first-reaction method. The first-reaction method determines when each reaction in the system would fire *if it were the only reaction present in the system* and then chooses τ as the smallest of these values and μ as the corresponding reaction. Such “tentative” next-reaction times are calculated via

$$\tau_{\mu}^{\text{exact}} = -\ln(r_{\mu})/a_{\mu}(t), \quad (1)$$

where r_{μ} is a unit-uniform random number. As originally formulated, the first-reaction method requires M unit-uniform random number generations at each simulation step, $M - 1$ of which are discarded before proceeding on to the next step. An improvement upon this approach is Gibson and Bruck's next-reaction method (NRM).⁵¹ The NRM basically uses a rigorous random-variable transformation formula to reuse the generated random numbers in the next time step. This reduces the number of random number generations per time step to exactly one, along with $M' - 1$ calculations of

$$\tau_{\mu}^{\text{exact}} = (a'_{\mu}(t)/a_{\mu}(t)) (\tau_{\mu}^{\text{exact}'} - \tau'), \quad (2)$$

where the unprimed and primed quantities signify new and old values, respectively.



Leonard's explanation of S-PLA



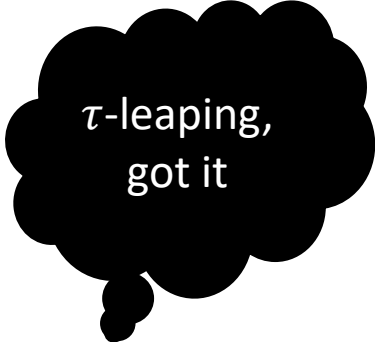
B. Leaping approaches

1. τ leaping

As mentioned previously, the primary shortcoming of exact-stochastic simulation methods, whether applied to well-mixed systems or otherwise, is that every event firing is simulated explicitly. This imposes a tremendous computational burden on the algorithm, particularly if one or more species have large populations.

To address this problem, Gillespie¹⁵ proposed the τ -leaping approach, which proceeds by firing multiple reaction events at each simulation step. In the well-mixed case, we first define the random variable $K_\mu(\tau)$ as the number of times reaction channel R_μ fires during the time interval $[t, t + \tau)$. The time evolution of the system can be formally written in terms of this variable as

$$\mathbf{X}(t + \tau) = \mathbf{X}(t) + \sum_{v=1}^M \mathbf{z}_v K_v(\tau). \quad (3)$$



τ -leaping,
got it

Leonard's explanation of S-PLA



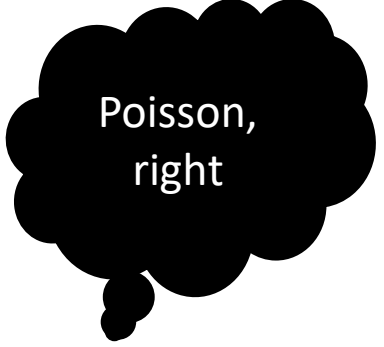
The idea then is to calculate some τ over which all reaction propensities remain “essentially constant.” In such a case, the reaction dynamics can be assumed to obey *Poisson statistics* and

$$K_{\mu}(\tau) \approx \mathcal{P}(a_{\mu}(t)\tau), \quad (4)$$

where $\mathcal{P}(a_{\mu}(t)\tau)$ is a Poisson random variable with mean and variance $a_{\mu}(t)\tau$. Note that the dependence in Eq. (4) on the value of a_{μ} at the beginning of the step, i.e., at the initial time t , makes this an “explicit” approach, analogous to explicit methods used in the numerical integration of ODEs.¹⁷

Equations (3) and (4) constitute the essence of the (explicit) τ -leaping method. At each step of a simulation, a time step τ is calculated (see Sec. ??? below) and the system state updated by generating M Poisson random deviates $\{k_v(\tau)\}$ in keeping with Eq. (4). Added to this is a *proviso* that if the total number of expected firings, $a_0(t)\tau$, is “small” (~ 10) then some variant of the SSA is used instead.¹⁵

Since its inception, modifications to the τ -leaping approach have been proposed by various investigators.^{16–31} There are recent reviews by Gillespie¹³ and Pahle.⁵⁴ Although differing in various aspects, all of these methods are based on the same basic principles encapsulated in Eqs. (3) and (4).



Poisson,
right

Leonard's explanation of S-PLA



Noise
negligible →
ODE, ok

2. Partitioned leaping

In Refs. 15 and 55, Gillespie went beyond Eq. (4) and noted a well-known property of the Poisson distribution that it can be approximated by a *normal*, or *Gaussian*, distribution if the mean is “large.” This allows us to write

$$K_{\mu}(\tau) \approx \mathcal{P}(a_{\mu}(t)\tau) \approx \mathcal{N}(a_{\mu}(t)\tau, a_{\mu}(t)\tau) = a_{\mu}(t)\tau + \sqrt{a_{\mu}(t)\tau} \times \mathcal{N}(0, 1), \quad (5)$$

where $\mathcal{N}(0, 1)$ is a normal random variable with mean zero and unit variance.^{15,55} Written this way, Eq. (5) is equivalent to the chemical Langevin equation,⁵⁵ a stochastic differential equation comprised of a “deterministic” term and a fluctuating “noise” term. Gillespie then noted that as $a_{\mu}(t)\tau \rightarrow \infty$ the noise term becomes negligible relative to the deterministic term, giving

$$K_{\mu}(\tau) \approx a_{\mu}(t)\tau, \quad (6)$$

which is equivalent to the forward-Euler method for solving deterministic ODEs.^{15,55}

In Ref. 31, we introduced the PLA, a τ -leaping variant that utilizes the entire theoretical framework encompassed by Eqs. (4)–(6). The PLA considers reactions individually in a way reminiscent of the NRM. After calculating a time step τ (Sec. ???), each reaction is *classified* into one of four categories: exact-stochastic, Poisson, Langevin, and deterministic. Reactions classified at the three coarsest levels (Poisson, Langevin, and deterministic) utilize Eqs. (4)–(6), respectively. Reactions classified at the exact-stochastic level are handled as in the NRM [Eqs. (1) and (2)]. Incorporating the SSA into the multiscale framework of the PLA is thus seamless and simple. Details of the algorithm can be found in Ref. 31, with a demonstration of its utility in Ref. 56.

Poisson to
Normal →
Langevin, fine

PLA →
4 regimes

Leonard's explanation of S-PLA



Uhm, ok...

4. Spatial τ -leaping

Spatial leaping approaches involve grouping events (reaction and diffusion) by subvolume, calculating a characteristic time interval τ_l^{leap} for each subvolume and then choosing the global time step

$$\tau = \min_{l' \in \{1, \dots, L\}} \{ \tau_{l'}^{\text{leap}} \} . \quad (8)$$

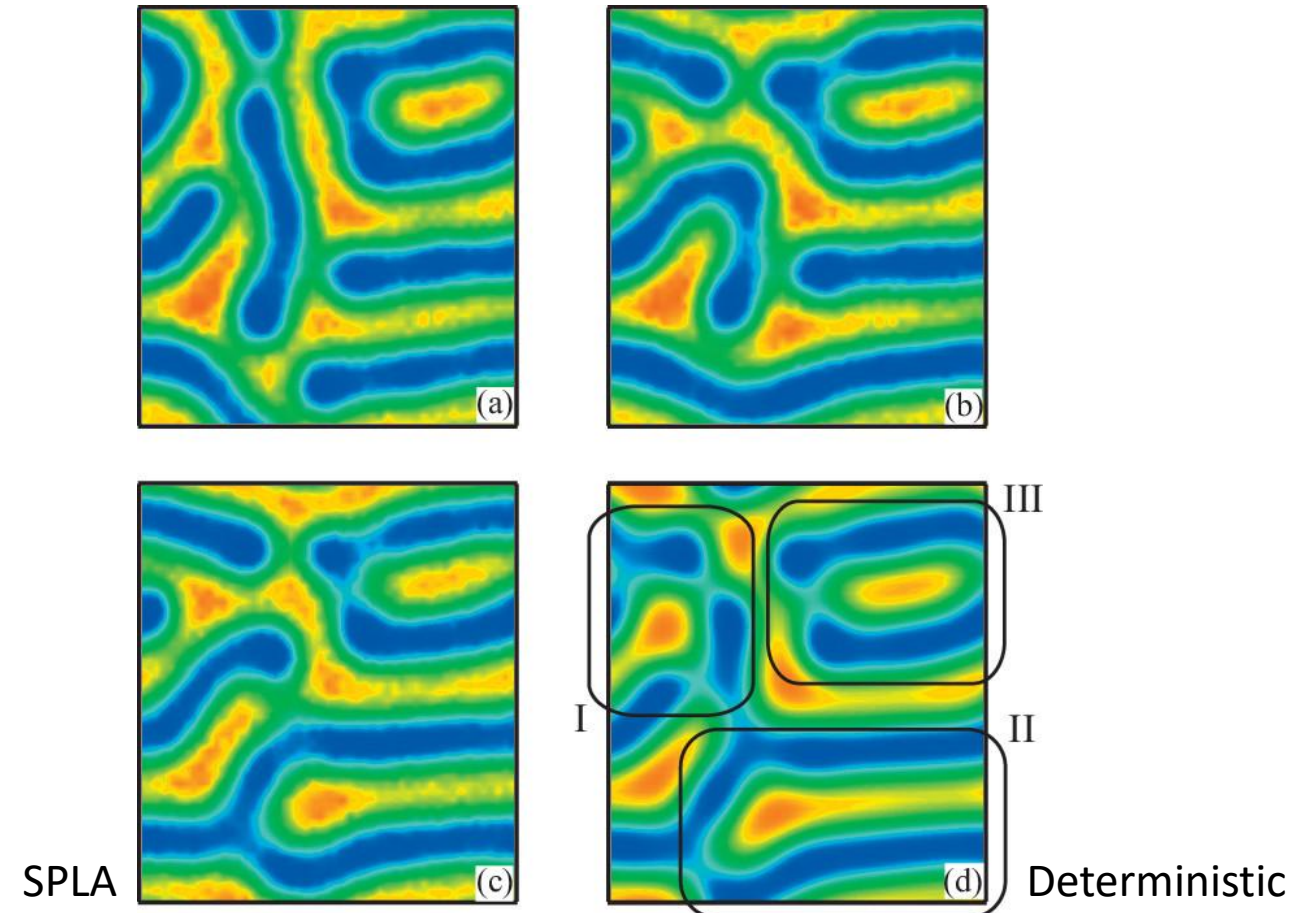
Every reaction and diffusion event can then fire multiple times within τ .

A. Motivation

A major concern with the Marquez-Lago and Burrage method is the exclusion of incoming diffusion events in the τ -selection process. In the NSM, incoming diffusion can be ignored when selecting values of τ because events outside the subvolume have no bearing on when the next event within the subvolume will fire. In leaping methods, however, this is no longer the case: The relationships between events are of central importance in selecting values of τ . Ignoring incoming diffusion in τ selection is thus an error that may impact the accuracy and/or efficiency of the method to an *a priori* indeterminable extent. In Sec. IV, we will show

In SPLA, we address each of the above issues: (i) both incoming and outgoing diffusions are taken into account in the τ -selection process, (ii) reactions and diffusion events are considered together when selecting time steps, (iii) appropriately modified formulations of the RB- a_μ and SB- a_μ τ -selection procedures are used, and (iv) the method automatically segues to an exact-stochastic method (NRM) at low populations.

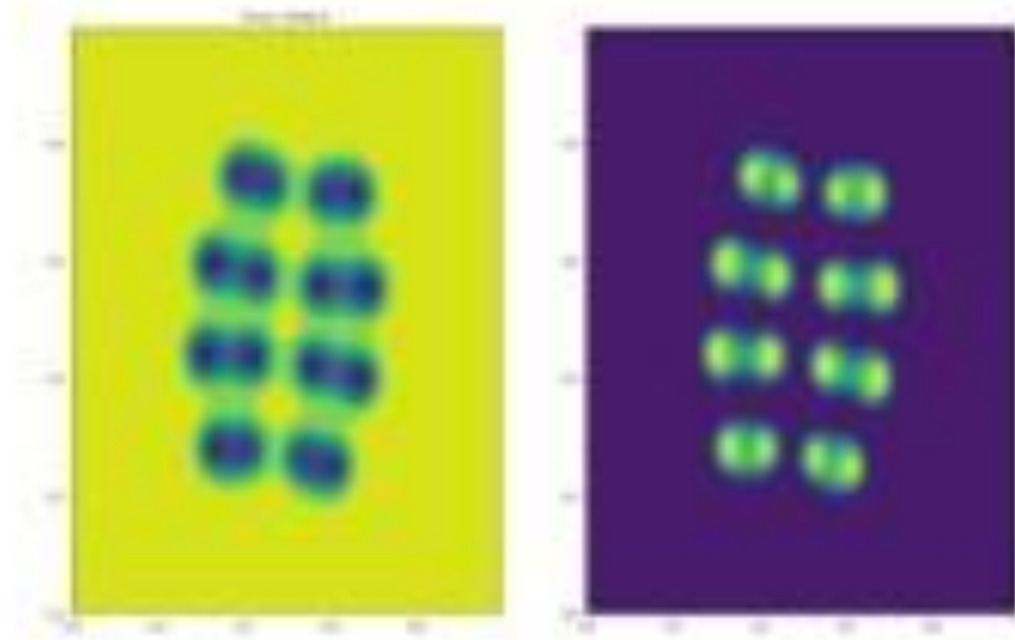
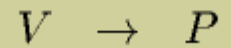
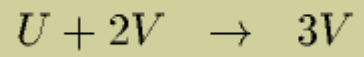
Example of S-PLA: Gray-Scott equations



Grey-Scott diffusion w/ periodic conditions



Chemical Reaction:



<https://www.youtube.com/watch?v=ESwQLcoCc8Y>

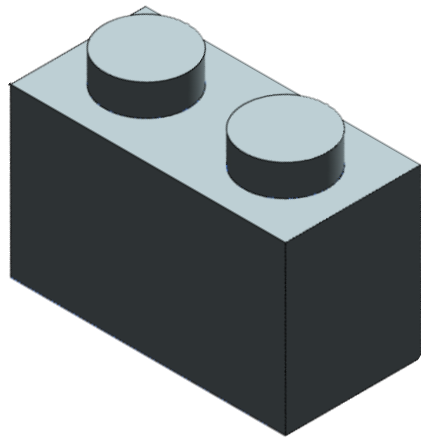
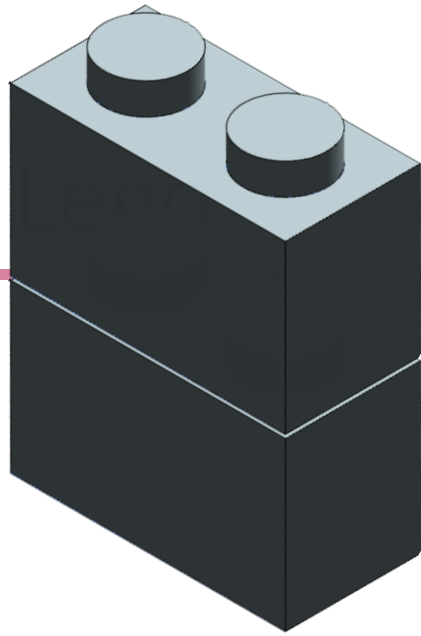
One final comment...



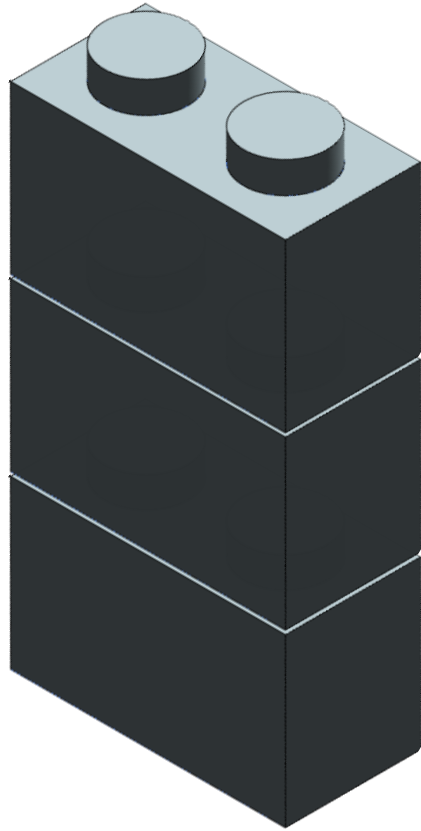
- Imagine that you want to model and simulate a very simple system that has a **SINGLE type of component**
- This component has **two regions that can bind** in a very specific way
- Do you need a concrete example? Imagine this **Lego brick**:



Rules of



Rules of Lego



- What are the "reactions" of this system?
- How many reactions do we need?
- Are we really sure we have a single species?
- What does the model look like?

Can the Lego system be modeled?



- It turns out that the number of "chemical species" and "chemical reactions" for this system is **infinite**
- We **cannot model** this system using a reaction-based approach
 - We need a more "expressive" way to represent reactions
- **Rule-based modeling**: describe the **binding sites** of chemical species and keep track of the bindings

Another example: EGFR family of proteins

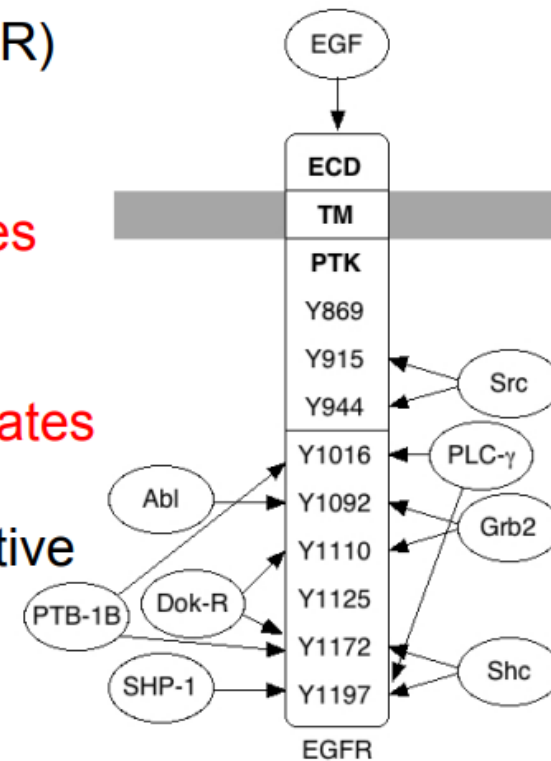


Epidermal growth factor receptor (EGFR)

9 sites $\Rightarrow 2^9 = 512$ phosphorylation states

Each site has ≥ 1 binding partner
 $\Rightarrow$ more than $3^9 = 19,683$ total states

EGFR must form *dimers* to become active
 $\Rightarrow$ more than 1.9×10^8 states



The slides that follow are taken from Bill Hlavacek's lectures at the Los Alamos National Laboratory of Theoretical Biology...

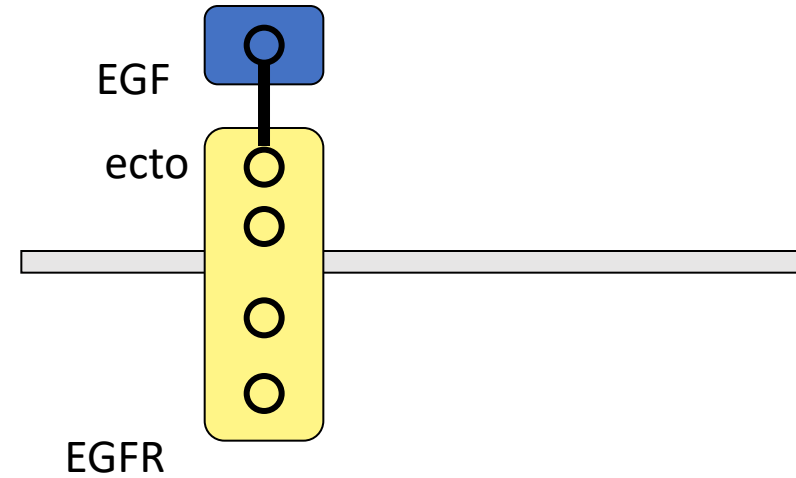
Early events in EGFR signaling



EGF = epidermal growth factor

EGFR = epidermal growth factor receptor

1. EGF binds EGFR

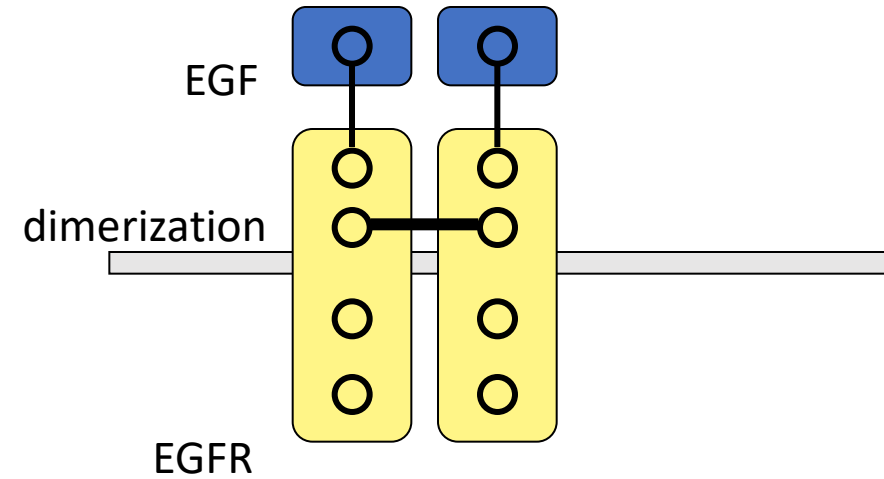


Early events in EGFR signaling



1. EGF binds EGFR

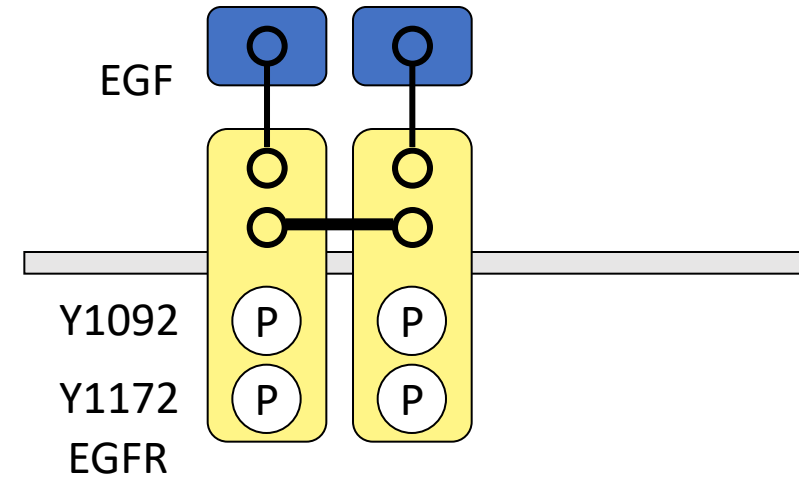
2. EGFR dimerizes



Early events in EGFR signaling



1. EGF binds EGFR
2. EGFR dimerizes
- 3. EGFR transphosphorylates itself**

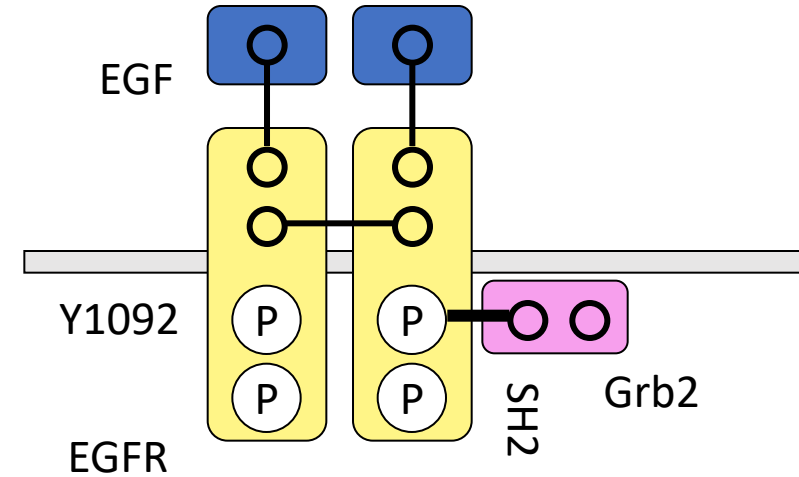


Early events in EGFR signaling



Grb2 pathway

1. EGF binds EGFR
2. EGFR dimerizes
3. EGFR transphosphorylates itself
4. **Grb2 binds phospho-EGFR**

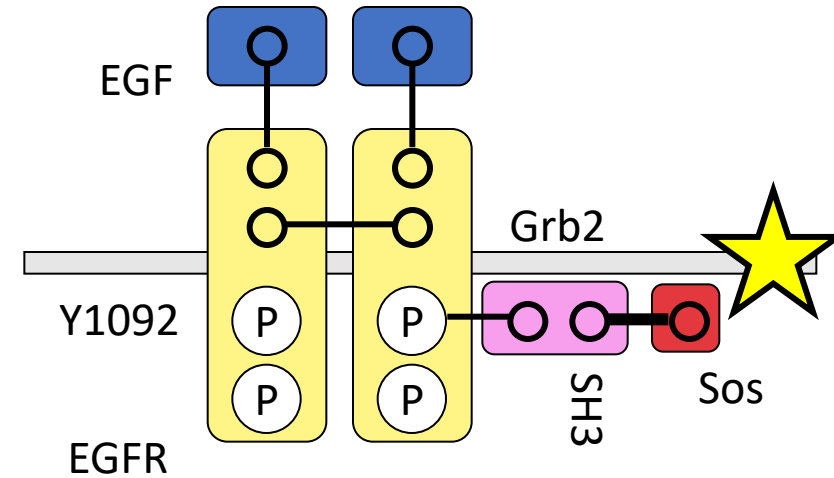


Early events in EGFR signaling



Grb2 pathway

1. EGF binds EGFR
2. EGFR dimerizes
3. EGFR transphosphorylates itself
4. Grb2 binds phospho-EGFR
5. **Sos binds Grb2 (Activation Path 1)**

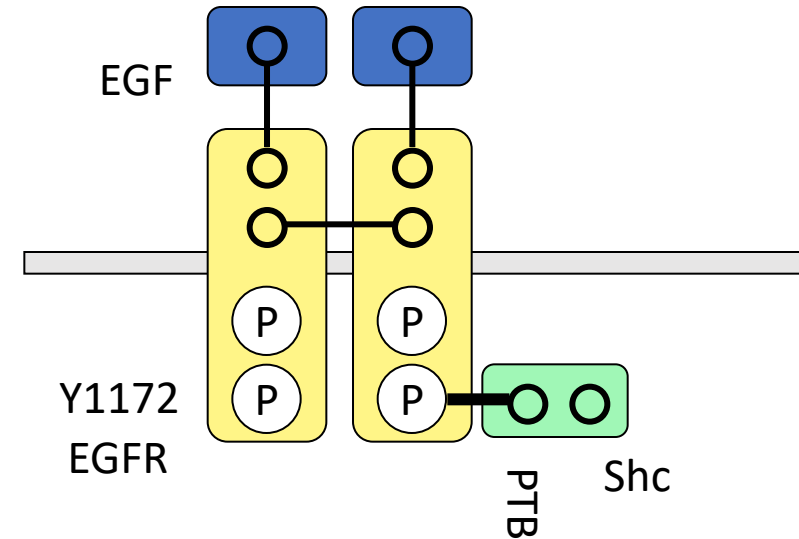


Early events in EGFR signaling



Shc pathway

1. EGF binds EGFR
2. EGFR dimerizes
3. EGFR transphosphorylates itself
- 4. Shc binds phospho-EGFR**

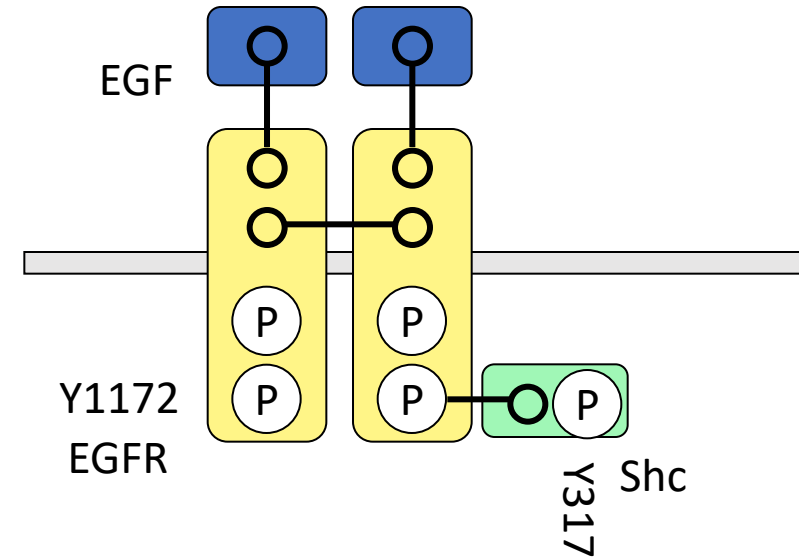


Early events in EGFR signaling



Shc pathway

1. EGF binds EGFR
2. EGFR dimerizes
3. EGFR transphosphorylates itself
4. Shc binds phospho-EGFR
- 5. EGFR transphosphorylates Shc**

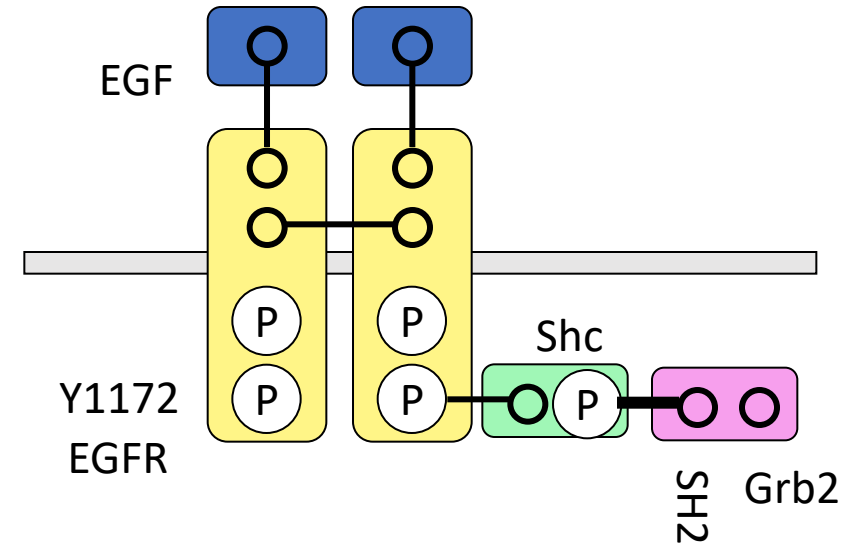


Early events in EGFR signaling



Shc pathway

1. EGF binds EGFR
2. EGFR dimerizes
3. EGFR transphosphorylates itself
4. Shc binds phospho-EGFR
5. EGFR transphosphorylates Shc
6. **Grb2 binds phospho-Shc**

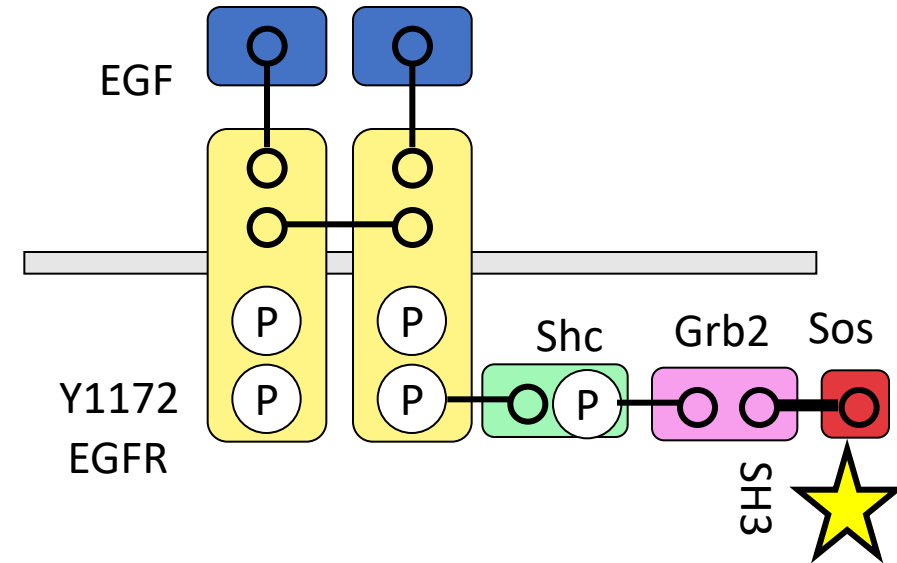


Early events in EGFR signaling



Shc pathway

1. EGF binds EGFR
2. EGFR dimerizes
3. EGFR transphosphorylates itself
4. Shc binds phospho-EGFR
5. EGFR transphosphorylates Shc
6. Grb2 binds phospho-Shc
7. **Sos binds Grb2 (Activation Path 2)**



Proteins are introduced with molecule templates



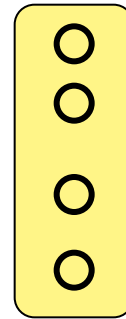
Molecule templates

EGF

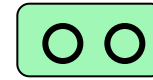


EGFR

L1
CR1
Y1092
Y1172

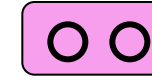


Shc



PTB
Y317

Grb2



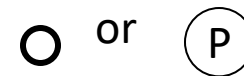
SH2
SH3

Sos



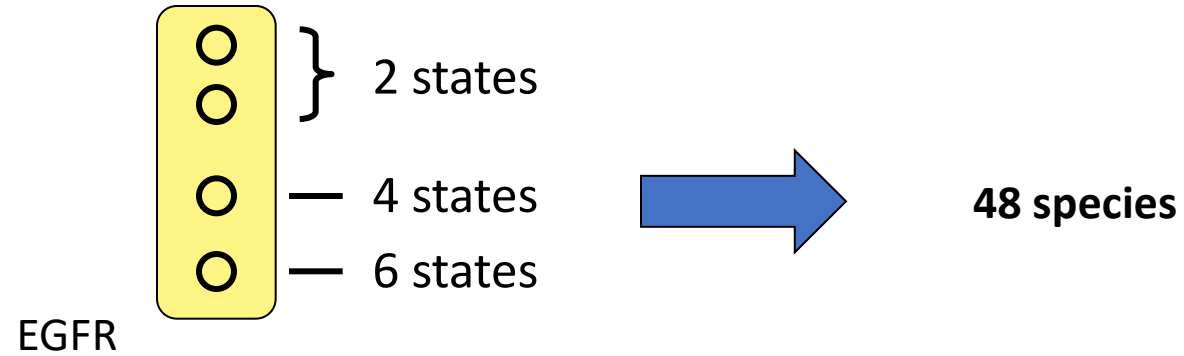
Nodes represent components of proteins

Components may have attributes:



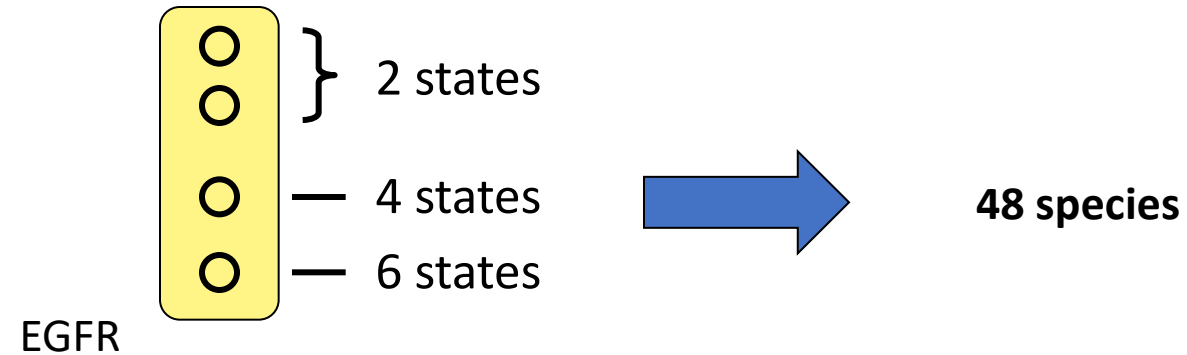
Combinatorial complexity of early events

Monomeric species

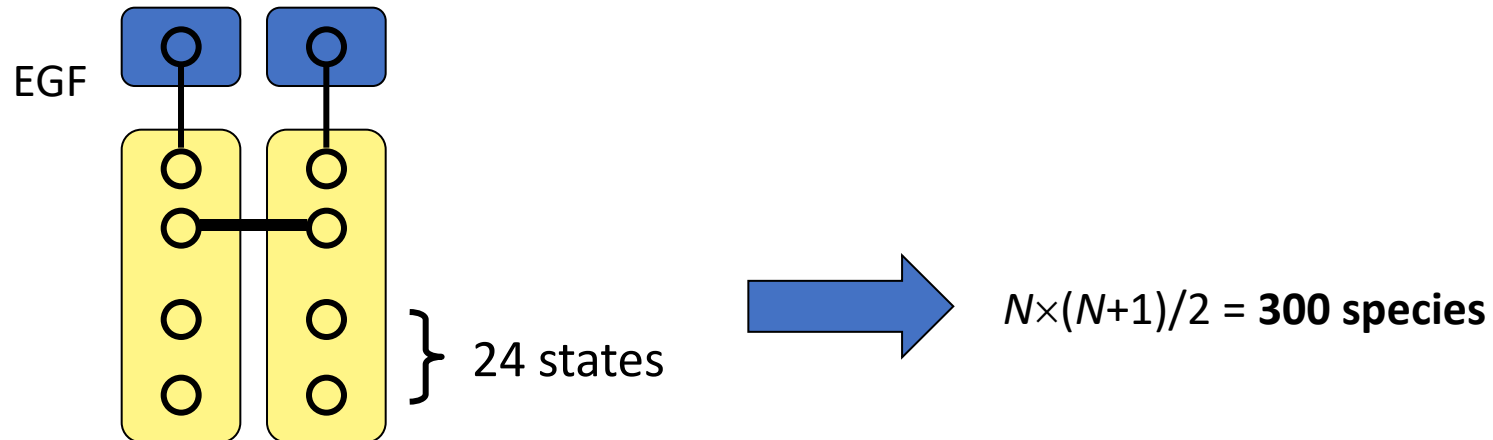


Combinatorial complexity of early events

Monomeric species



Dimeric species



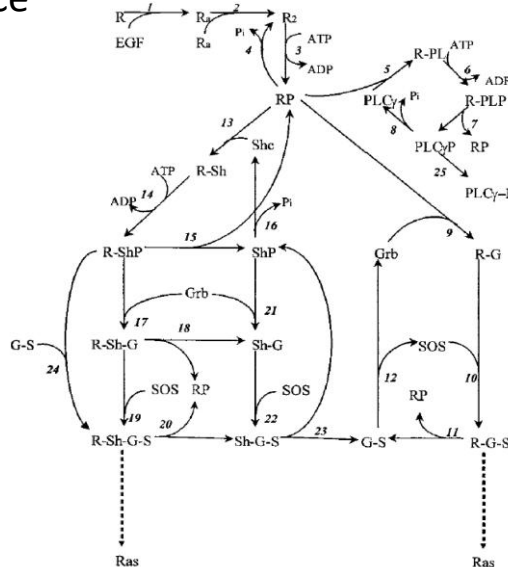
Rule-based version of the model



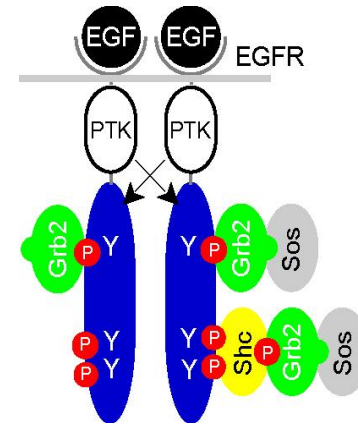
Blinov et al. *Biosystems* **83**, 136 (2006).

Original model,
contradicted by
experimental evidence

18 species
34 reactions



356 species
3749 reactions



- 5 molecule types
- 23 reaction rules
- No new rate parameters (!)

Network-free simulation



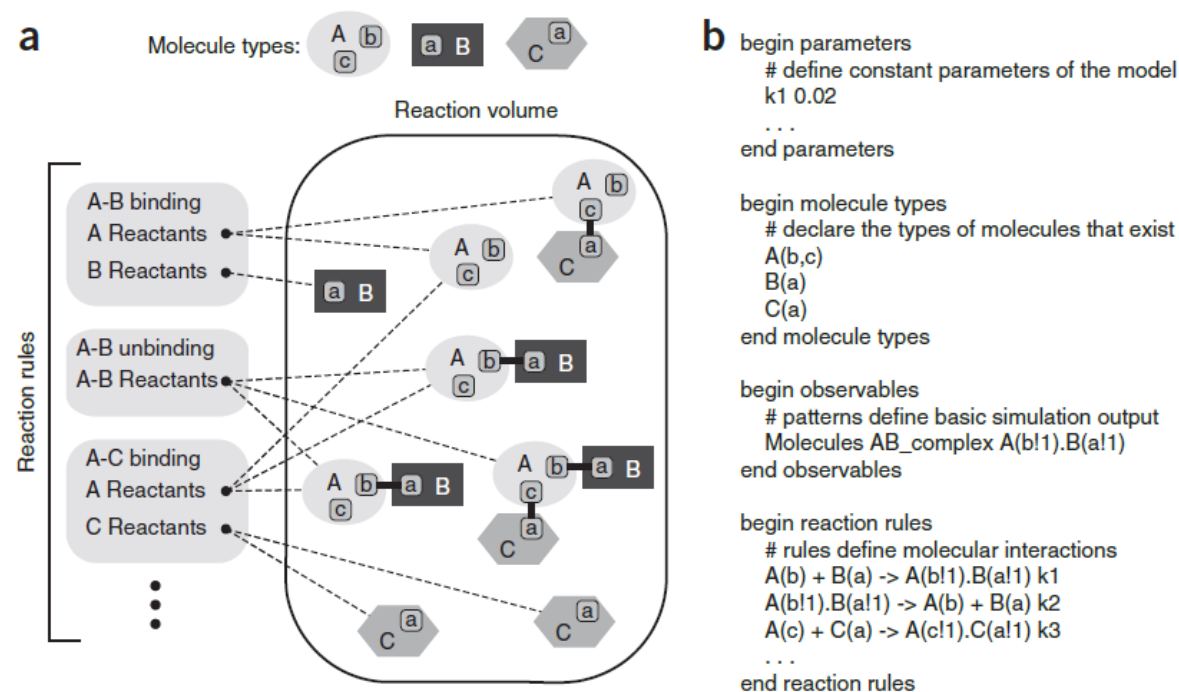
- **How can we simulate** a model that generates new species (and reactions!) during the simulation itself?
- The answer is **network-free simulation** (e.g., NFsim)

Network-free simulation

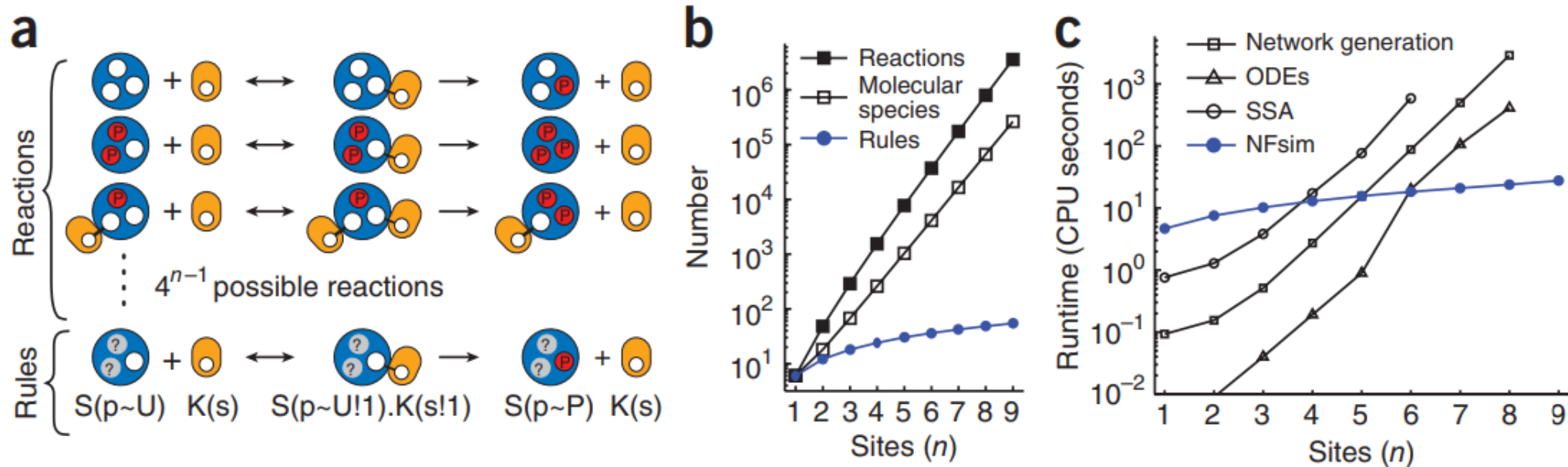


Figure 2 | Schematic overview of NFsim.

(a) Rules keep track of the molecular agents that can participate in a reaction (dashed lines) by matching possible reactants to user-defined reactant patterns. At each simulation step, a rule event is generated, reactant molecules are selected and transformed, and the set of possible reactants for each rule is updated (see Online Methods). In addition to the binding and unbinding events, rules can also specify reversible reactions, domain state changes, molecular synthesis and molecular degradation. (b) Example BNGL file specifying the model depicted in a. Lines that begin with # are user comments ignored by NFsim. Parameters and rate constants are defined in the parameters block. Molecule types 'A', 'B' and 'C' are defined with a set of labeled domains 'a', 'b' and 'c'. Observables specify molecular patterns that provide simulation output, such as the pattern 'AB_complex'. Molecular domains are referenced in rules to define how molecules interact. 'A(b!1).B(a!1)' denotes that a bond connects domain 'b' of molecule 'A' to domain 'a' of molecule 'B'. Here binding of 'A' to 'B' is declared to be independent of the binding of 'A' to 'C', because in the reactant pattern of the rule, the site named 'c' of molecule 'A' is omitted. Reaction rates are given as parameters 'k1', 'k2' and 'k3'.



Performance of rule-based, network-free



Summary



- When we deal with real world models, there is not a *one-size-fits-all* modeling approach
- On the contrary, there are some parts of the model that must be stochastic, and some other parts that could be more coarse-grained
- Hybrid simulators partition the system into slow and fast regimes, which are simulated using different methods (e.g., SSA vs ODE)
- Static partitioning is generally deemed to fail
- Harris' partitioned leaping algorithm (PLA) is the most advanced and comprehensive method, encompassing all regimes: SSA, tau-leap, CLE, and ODE
- A spatial version of PLA exist and is more accurate than competitors
- Rule-based and network-free simulation is more expressive than reaction-based modeling, and allow us to scale even further



Simulating SDEs [1/3]

- ODEs have this general form: $\frac{dX(t)}{dt} = F(X(t), t)$
 - That is, $dX(t) = F(X(t), t)dt$
- For instance, we can consider: $\frac{dX(t)}{dt} = a(t)X(t)$
- When $a(t)$ is not deterministic, we obtain a stochastic DE with parameter

$$a(t) = F(t) + G(t)\xi(t)$$

where $\xi(t)$ denotes a white noise process



Simulating SDEs [1/3]

- Remember that ODEs have this general form: $\frac{dX(t)}{dt} = F(X(t), t)$
 - That is, $dX(t) = F(X(t), t)dt$

- For instance, we can consider: $\frac{dX(t)}{dt} = a(t)X(t)$

- When $a(t)$ is not deterministic, we obtain a stochastic DE with parameter

$$a(t) = \underbrace{F(t)}_{\text{deterministic component}} + \underbrace{G(t)\xi(t)}_{\text{stochastic component}}$$

where $\xi(t)$ denotes a white noise process

Simulating SDEs [2/3]

- If we combine $\frac{dX(t)}{dt} = a(t)X(t)$ and $a(t) = F(t) + G(t)\xi(t)$ we get

$$\frac{dX(t)}{dt} = F(t)X(t) + G(t)\xi(t)X(t)$$

- If we rewrite the equation in **differential form**, it becomes

$$dX(t) = F(t)X(t)dt + G(t)\xi(t)X(t)dt$$



Simulating SDEs [3/3]

- We can set $dW(t) = \xi(t)dt$ where $dW(t)$ denotes a **Wiener process**, which defines an independent distribution of **perturbing values** at each time t

$$dX(t) = F(t)X(t)dt + G(x)X(t)dW(t)$$

- A more general form is the following:

$$dX(t, \omega) = F(t, X(t, \omega))dt + G(t, X(t, \omega))dW(t, \omega)$$

where ω denotes the fact that X is a stochastic variable (s.t. $P(X(0, \omega) = X_0) = 1$)

The Itô form of SDEs

$$dX_t = dt \cdot F(X_t, t) + dW_t \cdot G(X_t, t)$$

- How do we simulate this? We can simulate a **discretized process**, i.e., perform m simulation steps (that is, with time steps $h = \frac{T_{max}}{m}$)

- The simplest simulation scheme for this type of situation is the **Euler-Maruyama**:

$$X_{(k+1)h} = X_{kh} + F(kh, X_{kh})h + G(kh, X_{kh})\sqrt{h}\mathcal{N}(0,1)$$

where $\mathcal{N}(0,1)$ denotes IID samples from a **standard Gaussian** distribution

EM algorithm



```
for  $j = 1$  to  $n$ 
   $t = 0$ ;  $\hat{X} = X_0$ 
  for  $k = 1$  to  $\lfloor T/h \rfloor =: m$ 
    generate  $Z \sim N(0, 1)$ 
    set  $\hat{X} = \hat{X} + \mu(t, \hat{X})h + \sigma(t, \hat{X})\sqrt{h} Z$ 
    set  $t = t + h$ 
  end for
  set  $f_j = f(\hat{X})$ 
end for
set  $\hat{\theta}_n = (f_1 + \dots + f_n)/n$ 
set  $\hat{\sigma}_n^2 = \sum_{j=1}^n (f_j - \hat{\theta}_n)^2 / (n - 1)$ 
set approx.  $100(1 - \alpha) \%$  CI  $= \hat{\theta}_n \pm z_{1-\alpha/2} \frac{\hat{\sigma}_n}{\sqrt{n}}$ 
```

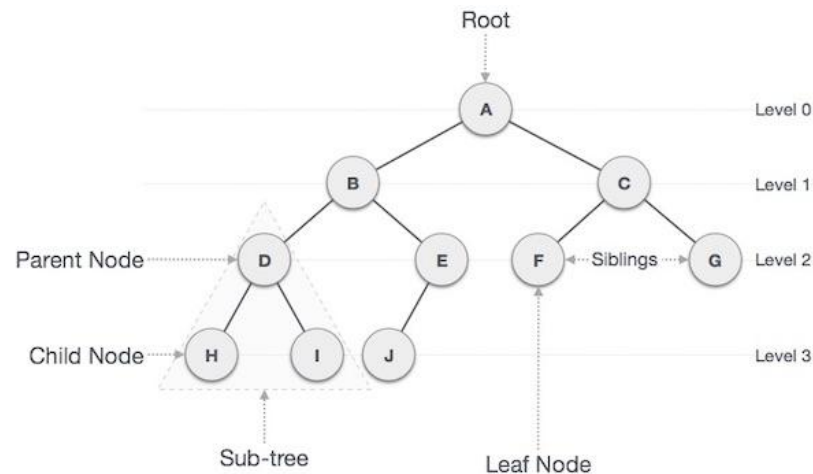
Summary of the first part



- Stochastic simulation can be computationally intensive, because it calculates one single reaction at a time
- We can introduce some assumptions and exploit probability theory to create more advanced algorithms (e.g., tau-leaping)
- However, there are some potential issues to be tackled
- Further assumptions make us transition from the discrete-stochastic regime towards the continuous-stochastic, and eventually deterministic

Binary trees

- A **binary tree** is a fundamental data structure in computer science
- It is a tree structure, so it is composed of **nodes** connected by edges
- Nodes are generally used to store a **value**
- The structure begins with **the topmost node named the "root"**
- Each node can have **up to 2 children** named "left" and "right" child
- Nodes without children take the name of "**leaves**"



HOW NORMAL PEOPLE SEE TREES

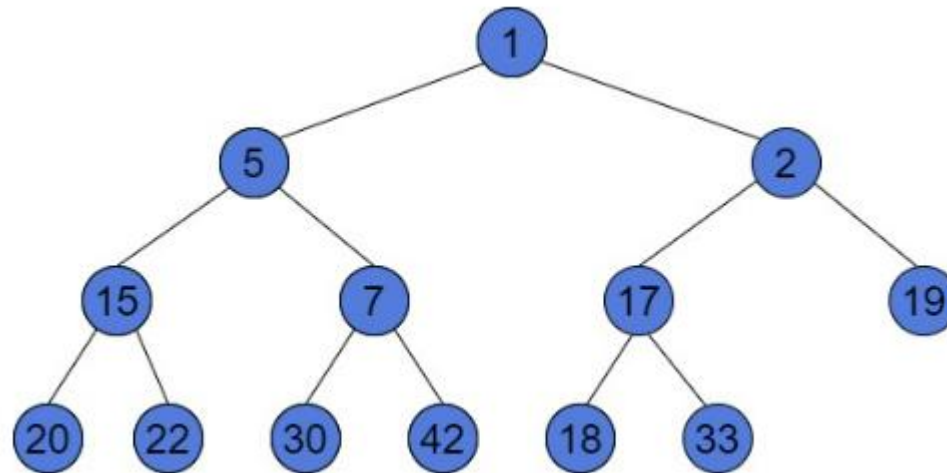


HOW COMPUTER SCIENTISTS SEE TREES



Heap

- Imagine now to have a **complete binary tree** (i.e., a tree in which each level except to last one must be filled with nodes)
- A **(min)heap** is a complete binary tree such that **the values stored by children are always larger than the values of the parents**



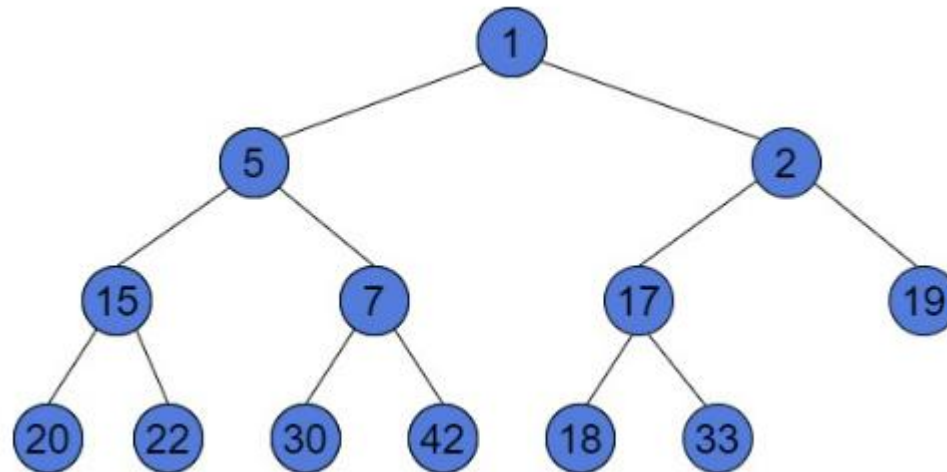
Is this a
binary tree?

Is it complete?

Is it a heap?

Heap

- Imagine now to have a **complete binary tree** (i.e., a tree in which each level except to last one must be filled with nodes)
- A **(min)heap** is a complete binary tree such that **the values stored by children are always larger than the values of the parents**



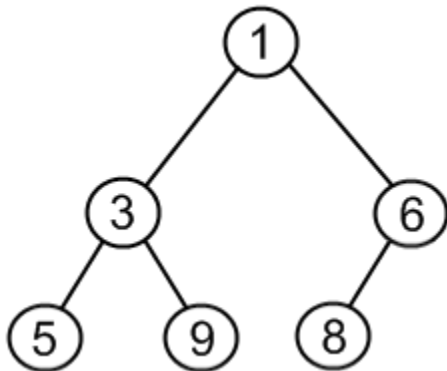
Is this a
binary tree?

Is it complete?

Is it a heap?

Adding a value to a heap

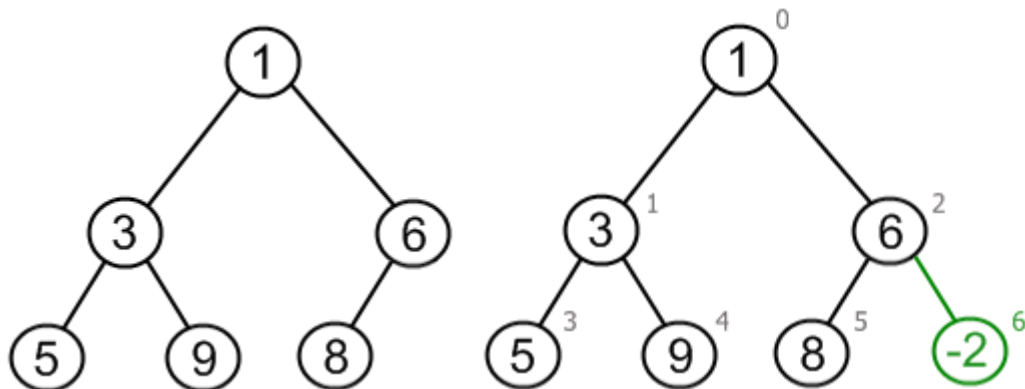
- As a first step to add a new value to the PT, we need to find the first **empty slot** for a new leaf and **create a new node** for that value
- Then, we **recursively check whether the heap condition is satisfied**, i.e., if the node is smaller than the parents
 - If the parent is larger than the node, then we **swap the two nodes** and repeat
 - Else, we are done
- Let's try to add the number -2 to the following heap:





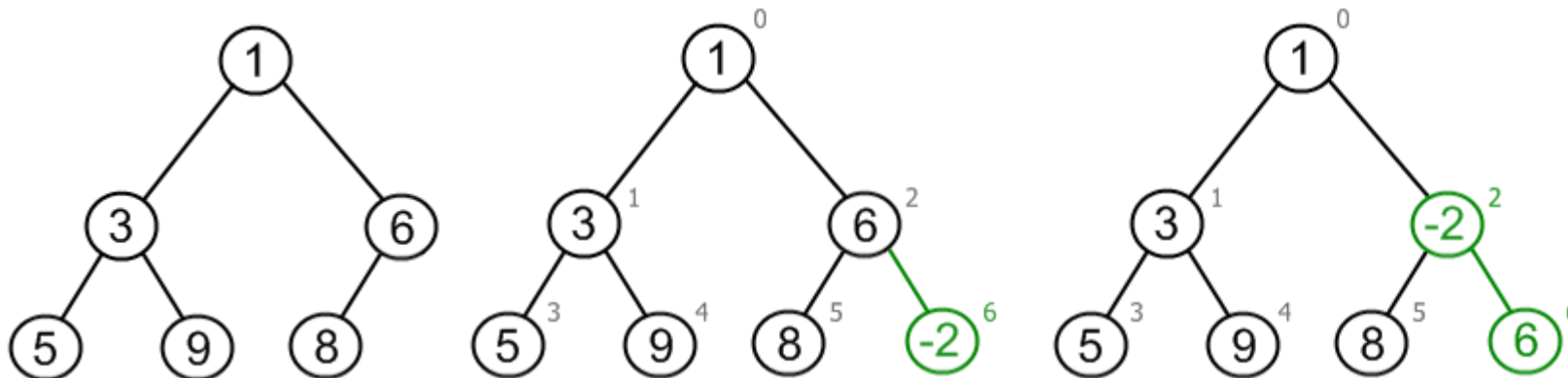
Adding a value to a heap

- As a first step to add a new value to the PT, we need to find the first **empty slot** for a new leaf and **create a new node** for that value
- Then, we **recursively check whether the heap condition is satisfied**, i.e., if the node is smaller than the parents
 - If the parent is larger than the node, then we **swap the two nodes** and repeat
 - Else, we are done
- Let's try to add the number -2 to the following heap:



Adding a value to a heap

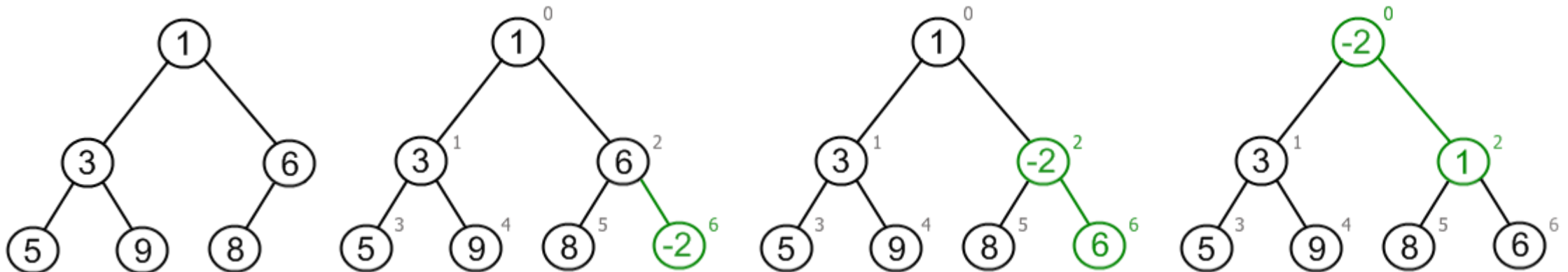
- As a first step to add a new value to the PT, we need to find the first **empty slot** for a new leaf and **create a new node** for that value
- Then, we **recursively check whether the heap condition is satisfied**, i.e., if the node is smaller than the parents
 - If the parent is larger than the node, then we **swap the two nodes** and repeat
 - Else, we are done
- Let's try to add the number -2 to the following heap:





Adding a value to a heap

- As a first step to add a new value to the PT, we need to find the first **empty slot** for a new leaf and **create a new node** for that value
- Then, we **recursively check whether the heap condition is satisfied**, i.e., if the node is smaller than the parents
 - If the parent is larger than the node, then we **swap the two nodes** and repeat
 - Else, we are done
- Let's try to add the number -2 to the following heap:





A couple of comments

- Observation #1: in a (min)heap, **the smallest element is always the root**
 - For free (free as in free beer!)
- Observation #2: the update of the heap requires just $\mathcal{O}(\log(n))$ **operations**
 - That is, it is very fast (sublinear!)
 - Update can be triggered by the **insertion** of a new node or the **change of one node's value**
- We can exploit these characteristics to **keep track** of the (fastest) **reactions** across all subvolumes
 - One node == one subvolume
 - After a reaction or diffusion occurs, we must update the propensities
 - Hence, we must recalculate the τ values for that subvolume and the affected neighbours
 - This is basically how NSM implements its **events queue**